الوعي الرقمي

المعرفة التي تحميك وتصنع مستقبلك



المعرفة قوة، لكن قيمتها الحقيقية
فيما نقدمه بها للآخرين

تأليف: جنكيز

الوعي الرقمي
المعرفة التي تحميك وتصنع مستقبلك
تأليف: جنكيز

كتاب تخصصي في الثقافة التقنية والأمن السيبراني

الوعي الرقمي

المعرفة التي تحميك وتصنع مستقبلك

تأليف: جنكيز

«المعرفة قوة، لكن قيمتها الحقيقية تتجلّى في نفع الآخرين بها».

الطبعة الأولى — 2026م / 1447هـ

صفحة حقوق النشر والبيانات الببليوغرافية

عنوان الكتاب: الوعي الرقمي: المعرفة التي تحميك وتصنع مستقبلك

المؤلف: جنكيز

لغة الكتاب: العربية الفصحى الحديثة

الطبعة: الطبعة الأولى — 2026م / 1447هـ

التصنيف الموضوعي: علوم الحاسب / الأمن السيبراني / البرمجة والتعليم التقني / الثقافة الرقمية

إشعار حقوق الملكية الفكرية

جميع الحقوق محفوظة للمؤلف © 2026 جنكيز.

لا يجوز نسخ أي جزء من هذا الكتاب، أو تخزينه في أي نظام استرجاع إلكتروني، أو نقله بأي شكل من الأشكال أو بأي وسيلة—إلكترونية، أو ميكانيكية، أو تصويرية، أو تسجيلية—بقصد الاستغلال التجاري دون إذن كتابي مسبق من المؤلف. يُسمح بالاقتباس العلمي القصير لأغراض المراجعة والتدريس والتوعية المجتمعية مع الإشارة الصريحة للمصدر.

تنويه إخلاء المسؤولية التعليمي والدفاعي (Security & Legal Disclaimer)

المعلومات والنماذج والأكواد البرمجية الواردة في هذا الكتاب مُعدة حصرياً لأغراض تعليمية، وتوعوية، وتثقيفية، ودفاعية لتعزيز الأمن الرقمي الشخصي والمؤسسي، وحماية الأنظمة والبيانات، ونشر الفكر التقني الإيجابي.

المؤلف والناشر يُخليان مسؤوليتهما القانونية والأخلاقية عن أي استخدام غير مشروع، أو غير مصرح به، أو أي محاولة لإساءة توظيف المفاهيم التقنية الواردة هنا لاختراق الأنظمة، أو التعدي على خصوصية الأفراد، أو انتهاك القوانين واللوائح التنظيمية لمكافحة الجرائم المعلوماتية. إن تطبيق أي تقييم أمني عملي يجب أن يتم حصرياً داخل بيئات افتراضية معزولة، أو على أنظمة يمتلك المستخدم إذناً خطياً ورسمياً مسبقاً باختبارها.

الإهداء

إلى كل إنسان يسعى لفهم هذا الفضاء الرقمي، ويحرص على حماية أسرته ومجتمعه من مخاطره... إلى الشباب والمبرمجين وصنّاع الغد، الذين يحملون شغف التعلّم، ويؤمنون بأن الأسطر البرمجية ليست مجرد أوامر لآلة، بل هي أدوات للبناء وصناعة الأثر... وإلى كل من جعل من علمه جسراً يرفعه الآخرون، تذكيراً بالرسالة الراسخة: **«المعرفة قوة، لكن قيمتها الحقيقية تتجلّى في نفع الآخرين بها».** أهدي هذا العمل.

كلمة المؤلف: جنكيز

بسم الله الرحمن الرحيم،

نعيش اليوم في عالم أصبحت فيه التكنولوجيا جزءًا أساسيًا من حياتنا اليومية. فنحن نستخدمها للتواصل مع الآخرين، والتعلّم، والعمل، وإدارة شؤوننا المختلفة، كما نعتمد عليها في حفظ معلوماتنا وذكرياتنا.

ومع الفوائد الكبيرة التي تقدمها التكنولوجيا، تظهر تحديات جديدة تتعلق بحماية الخصوصية وأمن المعلومات والاستخدام المسؤول للأدوات الرقمية.

ومن هنا جاءت فكرة هذا الكتاب؛ ليكون دليلًا يساعد القارئ على فهم العالم الرقمي، والتعرف إلى مخاطره، واكتساب المعرفة اللازمة للتعامل معه بوعي وأمان.

لم أرد لهذا العمل أن يكون سردًا نظريًا جافًا يعقّد المفاهيم على القارئ العادي، ولا استعراضًا سطحيًا لا يروي شغف المهتم بالتقنية. لقد حرصت على أن أقدّم لك المعرفة من منبعها العملي؛ لترى ما يجري خلف الشاشات بوضوح، وتدرك كيف تفكر الأنظمة، وكيف يتربص المخترقون بالثغرات، وكيف تحمي نفسك وعائلتك دون خوف أو إفراط.

إن الأمان الرقمي في جوهره ليس مجرد أدوات نثبّتها، بل هو نمط تفكير وقرارات يومية واعية. وإذا كانت البرمجة تمكّننا من بناء أنظمة مذهلة، فإن وعينا الإنساني ومسؤوليتنا الأخلاقية هما الضمان الوحيد لأن تبقى هذه التقنيات في خدمة الإنسان وسعادته.

أضع هذا الكتاب بين يديك، آملاً أن تجد في صفحاته نفعاً حقيقياً يرشدك في مسيرتك الرقمية، ويسهم في بناء مجتمع عربي واعٍ ومحصّن قادر على صناعة مستقبله بثقة.

جنكيز

الفصل الأول: الإنسان في العصر الرقمي

> *«إن أعظم تحول في تاريخ البشرية الحديث لم يكن في بناء ناطحات السحاب أو غزو الفضاء، بل كان في بناء شبكة غير مرئية تربط عقول مليارات البشر وأجهزتهم في نبضة واحدة».*

1. افتتاحية: العالم الذي نستيقظ فيه كل صباح

تأمل معي مشهداً يتكرر مليارات المرات كل صباح في مختلف بقاع الأرض: يدق منبه الهاتف الذكي في غرفتك؛ تمد يدك لتوقفه بلمسة خفيفة. في تلك اللحظة بالذات، وقبل أن تغادر فراشك، تنظر إلى شاشة جهازك لتشاهد خريطة الطقس المحدثة لحظياً عبر الأقمار الاصطناعية، وتقرأ خبراً عاجلاً وقع قبل دقيقة واحدة في قارة أخرى، وتستلم رسالة صوتية من صديق يبعد عنك آلاف الكيلومترات، وتدفع فاتورة الكهرباء عبر تطبيقك البنكي، ثم تطلب وسيلة نقل تقلك إلى عملك وتتابع حركتها الدقيقة على الخريطة عبر إشارات نظام تحديد المواقع العالمي (GPS).

كل هذا يحدث في أقل من خمس دقائق، دون أن تلتفت إلى المعجزة العلمية والهندسية التي جعلت ذلك ممكناً.

لقد أصبح العالم الرقمي هو الفضاء الطبيعي الثاني للإنسان الحديث. نحن نأكل، ونتعلم، ونعمل، ونتواصل، ونحفظ ذكرياتنا، وندير مدخراتنا، ونعبر عن آرائنا عبر هذا الفضاء. ولكن، كم منا يعرف حقاً ما الذي يحدث خلف هذا الزجاج الأملس اللامع عندما ننقر بإصبعنا على الشاشة؟ كيف تنتقل أصواتنا وصورنا وأموالنا عبر الهواء؟ وما الثمن الذي ندفعه مقابل هذه الراحة المطلقة؟

إن الهدف من هذا الفصل الأول ليس تقديم سرد تاريخي ممل، بل **نزع الغموض عن العالم الرقمي**؛ لتبدأ رحلتك في هذا الكتاب وأنت تفهم الأرضية التي تقف عليها، وتدرك لماذا أصبح الوعي الرقمي طوق نجاة لا غنى عنه لكل إنسان.

---

2. ما هو التحول الرقمي وما مستوياته؟

كثيراً ما نسمع مصطلح **«التحول الرقمي» (Digital Transformation)** في الأخبار وخطابات الحكومات والشركات، ولكن ما معناه الدقيق بعيداً عن الرنين التسويقي؟

لنفهم التحول الرقمي، يجب أن نميز بين ثلاثة مستويات متراتبة:

1. **الرقمنة المادية (Digitization):**

هي تحويل المعلومات من صيغتها الفيزيائية التقليدية (ورق، صور

مطبوعة، أشرطة صوتية) إلى صيغة رقمية ثنائية تفهمها الحواسيب (أصفار وآحاد). مثال ذلك: مسح وثيقة ورقية ضوئياً وتحويلها إلى ملف PDF.

2. **رقمنة العمليات (Digitalization):**
هي توظيف التكنولوجيا الرقمية لتحسين إجراء أو معاملة معينة وجعلها أسرع وأكثر كفاءة. مثال ذلك: بدلاً من الوقوف في طابور طويل لتجديد رخصة القيادة، تقوم بملء نموذج عبر موقع إلكتروني.

3. **التحول الرقمي الشامل (Digital Transformation):**
هو تغيير جذري في نموذج التفكير والعمل والحياة، بحيث تصبح التكنولوجيا هي الأساس الذي يُعاد بناء المجتمع والخدمات والعلاقات وفقاً له. لم يعد الأمر مجرد «أتمتة» لما هو قديم، بل خلق أساليب حياة واقتصاد ومعرفة لم تكن ممكنة من قبل.

إن التحول الرقمي لا يعني استبدال الورق بالشاشات فحسب، بل يعني انتقال مركز الثقل الإنساني إلى عالم تتدفق فيه البيانات بسرعة الضوء.

---

3. كيف يعمل الإنترنت فيزيائياً؟ (حقائق خلف السحاب)

عندما نتحدث عن «الحوسبة السحابية» أو «رفع الملفات إلى الإنترنت»، يتخيل الكثير من الناس أن معلوماتهم تطفو حرفياً في الهواء الطلق أو بين الغيوم!

الحقيقة الصادمة لمعظم المبتدئين هي أن **الإنترنت شبكة فيزيائية ملموسة بنسبة تفوق 95%**.

إذا انقطعت الكابلات المادية، يتوقف الإنترنت فوراً.

كابلات الأعماق: شرايين الكوكب الرقمي

في هذه اللحظة التي تقرأ فيها هذه السطور، هناك ما يزيد عن **1.4 مليون كيلومتر** من كابلات الألياف الضوئية الممتدة في قيعان المحيطات والبحار، تربط القارات بعضها ببعض. هذه الكابلات ليست بحجم أنابيب النفط الضخمة، بل إن قطر الكابل في الأعماق لا يتجاوز خرطوم مياه الحديقة، ولكنه مغلف بطبقات فولاذية وعوازل تحميه من ضغط المياه وأسماك القرش والحركات التكتونية.

داخل هذا الكابل الصغير، توجد شعيرات زجاجية بالغة الدقة (أرق من شعرة الإنسان)، تنقل البيانات ليس ككهرباء، بل على شكل **ومضات ضوئية من الليزر (Light Pulses)** تسير بسرعة تقارب 200,000 كيلومتر في الثانية!

مراكز البيانات: قلاع المعلومات

أين تستقر صورك ورسائلك وحساباتك؟ إنها في مبانٍ خرسانية عملاقة تُعرف باسم **«مراكز البيانات» (Data Centers)**، تنتشر حول العالم في مناطق باردة ومؤمنة بأعلى درجات الحراسة وأنظمة التبريد ومولدات الطاقة الاحتياطية. تحتوي هذه المراكز على آلاف الرفوف الممتلئة بالحواسيب العملاقة والخوادم (Servers) التي تعمل دون انقطاع ثانية واحدة على مدار العام.

4. الفرق الحقيقي بين الإنترنت وشبكة الويب

يستخدم معظم الناس كلمتي «الإنترنت» و«الويب» وكأنهما مترادفان، لكن بينهما فرقاً علمياً جوهرياً يشبه الفرق بين **«شبكة الطرق السريعة» و«السيارات التي تسير عليها»**.

- **الإنترنت (The Internet):**

هو **البنية التحتية العالمية الهائلة**؛ أي الأسلاك، والكابلات البحرية، والأقمار الاصطناعية، والموجهات (Routers)، والخوادم، وأجهزة الحواسيب والهواتف المتصلة ببعضها وفق بروتوكولات اتصال موحدة. الإنترنت هو شبكة الشبكات التي تنقل الإشارات.

- **الويب (The World Wide Web - WWW):**

هو **أحد التطبيقات والخدمات التي تعمل فوق الإنترنت**. ابتكره العالم البريطاني «تيم بيرنرز لي» عام 1989، وهو منظومة الصفحات والمستندات المترابطة بواسطة الروابط التشعبية (Hyperlinks)، والتي تتصفحها باستخدام متصفح الويب (مثل Chrome أو Firefox أو Safari) عبر بروتوكول HTTP/HTTPS.

المعنى العملي:

عندما تتصفح موقعاً إلكترونياً، فأنت تستخدم الويب عبر الإنترنت. أما عندما تجري مكالمة صوتية عبر WhatsApp، أو تلعب لعبة فيديو أونلاين، أو ترسل بريداً عبر تطبيق خاص، فأنت تستخدم شبكة الإنترنت دون المرور بالويب بالضرورة!

---

5. ما هي البيانات الرقمية؟ رحلة من الصفر والواحد إلى الصورة الحية

كل ما تراه على شاشتك: من فيلم سينمائي فائق الجودة، إلى مقطع صوتي، أو رسالة نصية، أو عملية تحويل بنكي، يتحول في نهاية المطاف داخل دوائر السيليكون في جهازك إلى شيئين اثنين فقط: **(0) و (1)**.

لماذا 0 و 1؟

لأن الحواسيب كائنات كهربائية. أبسط وأكثر طريقة مضمونة لحفظ ونقل الإشارات دون تلف هي الاعتماد على حالتين فيزيائيتين للمفتاح الإلكتروني (الترانزستور):

* **0:** غياب التيار الكهربائي (أو شحنة منخفضة / مطفأ).
* **1:** وجود التيار الكهربائي (أو شحنة مرتفعة / مشتعل).

يُسمى هذا العنصر الأصغر **«البت» (Bit)**. وعندما نجمع 8 بتات معاً، نحصل على **«البايت» (Byte)**، وهو الحجم الكافي لتخزين حرف أبجدي واحد. ومنها تتصاعد الأحجام:

* **الكيلوبايت (KB):** ألف بايت تقريباً (صفحة نصية قصيرة).
* **الميجابايت (MB):** مليون بايت (أغنية صوتية أو صورة عالية الدقة).
* **الجيجابايت (GB):** مليار بايت (فيلم سينمائي بجودة عالية).
* **التيرابايت (TB):** ألف مليار بايت (مكتبة ضخمة تضم ملايين الكتب).

---

6. رحلة المعلومة: ماذا يحدث في أجزاء من الثانية عندما تفتح موقعاً؟

تخيل أنك فتحت متصفحك وكتبت: `www.example.com` ثم ضغطت على زر الإدخال (Enter). ما الذي يحدث خلال الـ 200 مللي ثانية التالية؟

1. **الترجمة عبر نظام أسماء النطاقات (DNS):**
   الحواسيب والشبكات لا تفهم الأسماء البشرية مثل `example.com` . إنها تتعامل مع عناوين رقمية تُعرف بـ **عنوان بروتوكول الإنترنت (IP Address)** مثل `93.184.216.34` .
   يقوم جهازك بالاتصال بدليل هاتف رقمي عالمي يُسمى **DNS (Domain Name System)** يسأله: "ما هو الـ IP التابع لهذا الاسم؟"، فيرد عليه في غمضة عين بالرقم الصحيح.

2. **تقطيع البيانات إلى حزم (Packets):**
   الإنترنت لا يرسل الصفحات والملفات ككتلة واحدة ضخمة. بدلاً من ذلك، تُقطع البيانات إلى مئات أو آلاف **«الحزم الصغيرة» (Packets)**. كل حزمة تحتوي على جزء من المحتوى، ملصقاً عليها عنوان المرسل وعنوان المستقبل ورقم تسلسلي.

3. **الارتحال عبر الموجهات (Routers):**
   تنطلق هذه الحزم عبر أسلاك الواي فاي إلى الراوتر المنزلي، ومنه إلى مزود الخدمة (ISP)، ثم تعبر القارات عبر الكابلات البحرية، متنقلة بين عشرات الموجهات (Routers) التي تقرر في كل ميكروثانية أي المسارات أسرع وأقل ازدحاماً.

4. **إعادة التجميع والمصافحة:**
تصل الحزم إلى خادم الموقع في ولاية أو دولة أخرى، فيقوم بتجميعها والتأكد من عدم ضياع أي حزمة، ثم يرسل صفحة الويب المطلوبة بنفس الطريقة إلى جهازك ليقوم متصفحك برسمها وتنسيقها أمام عينيك.

كل هذه الرحلة المعقدة الممتدة لآلاف الكيلومترات حدثت في لمح البصر وأنت ترمش عينك!

---

7. دور الخوادم والشبكات ومزودي الخدمة

لكي تتخيل شبكة الإنترنت بوضوح، فكر فيها كهيكل هرمي متماسك:

- **مزود خدمة الإنترنت (ISP - Internet Service Provider):**
هو الشركة المحلية التي تشتري منها اشتراك الإنترنت المنزلي أو باقة الهاتف المحمول. هذه الشركات بدورها تتصل بشركات اتصالات دولية عملاقة تملك الكابلات القارية والأقمار الصناعية (تسمى مشغلي المستوى الأول Tier-1).

- **العميل (Client):**
هو جهازك (هاتفك، حاسوبك الشخصي، جهازك اللوحي) الذي يطلب الخدمة أو الصفحة.

- **الخادم (Server):**
هو حاسوب عالي الكفاءة والمواصفات، متصل بالإنترنت طوال اليوم برابط

سريع جداً، دوره هو «خدمة» العملاء والاستجابة لطلباتهم بتسليمهم البيانات المخزنة لديه.

- **الموجه (Router):**

هو شرطي المرور الرقمي؛ يستلم حزم البيانات، ويقرأ وجهتها النهائية، ويوجهها إلى المسار السلكي أو اللاسلكي الأنسب دون أن تتصادم أو تضيع.

---

8. للقارئ المتخصص: نموذج TCP/IP ومفهوم التغليف (Encapsulation)

🔬 للقارئ المتخصص (Technical Deep Dive) تقوم شبكة الإنترنت عالمياً على معمارية معيارية موحدة هي **نموذج TCP/IP المكون من أربع طبقات رئيسية**: 1. **طبقة التطبيقات (Application Layer):** تتعامل مع البروتوكولات المباشرة للبرمجيات مثل HTTP/HTTPS لتصفح الويب، وDNS لترجمة الأسماء، وSMTP لنقل البريد، وSSH للاتصال الآمن عن بُعد. 2. **طبقة النقل (Transport Layer):** المسؤولة عن إدارة الاتصال بين البرامج من طرف لطرف. أشهر بروتوكولين فيها: * **TCP (Transmission Control Protocol):** موثوق، يتطلب تأسيس اتصال عبر المصافحة الثلاثية (3-Way Handshake: SYN -> SYN-ACK -> ACK)**، ويضمن وصول جميع الحزم بالترتيب مع إعادة إرسال الحزم المفقودة. * **UDP (User Datagram Protocol):** سريع وخفيف،

لا يتطلب مصافحة ولا يضمن الترتيب، وهو مثالي للبث الحي والصوتيات والألعاب الشبكية. 3. **طبقة الإنترنت (Internet / Network Layer):** تعتمد بروتوكول IP (بنسختيه IPv4 و IPv6) ومهمتها العنونة المنطقية والتوجيه (Routing) عبر مسارات الشبكات المختلفة. 4. **طبقة الوصول للشبكة (Link / Network Access Layer):** تتعامل مع العناوين الفيزيائية للأجهزة (MAC Addresses) وتحويل الإشارات إلى نبضات كهربائية عبر كابلات الإيثرنت أو موجات راديو (Wi-Fi) أو ضوئية في الألياف. #### مفهوم التغليف وفك التغليف (Encapsulation & Decapsulation): عندما يرسل تطبيقك رسالة، تهبط البيانات من أعلى إلى أسفل: * تضيف طبقة النقل ترويسة (Header) تضم أرقام المنافذ (Ports) لتصبح **Segment**. * تضيف طبقة الإنترنت ترويسة الـ IP (عناوين المصدر والوجهة) لتصبح **Packet**. * تضيف طبقة الربط ترويسة وتذييلاً بـ MAC Address لتصبح **Frame**. * تتحول إلى **Bits** في الوسط الفيزيائي. وعند الطرف المستقبل، تنعكس العملية (Decapsulation) صعوداً حتى تصل الرسالة نظيفة إلى التطبيق المعني.

---

9. لماذا أصبحت المعرفة الرقمية ضرورة وجودية؟

في القرون الماضية، كانت الأمية تعني عدم القدرة على القراءة والكتابة. أما في القرن الحادي والعشرين، فإن الأمية الحقيقية هي **الأمية الرقمية**.

إن الإنسان الذي يجهل كيف تعمل التكنولوجيا في محيطه يعيش في حالة استضعاف مستمر:

* **استضعاف أمام المراقبة والتتبع:** لا يدري من يجمع بياناته، ولا كيف يتم تصنيف اهتماماته وسلوكياته وبيعها لشبكات الإعلانات والتحليل.
* **استضعاف أمام الاستغلال والاحتيال:** ينخدع بسهولة برسائل التخويف أو وعود الثراء السريع، ويسلّم مفاتيحه وأمواله طواعية للمهاجمين.
* **استضعاف مهني ومستقبلي:** يفقد القدرة على المنافسة في سوق عمل يعيد تشكيل نفسه يومياً حول الأتمتة والذكاء الاصطناعي وإدارة البيانات.

الوعي الرقمي ليس ترفاً للمهووسين بالتكنولوجيا أو المتخصصين بالأمن، بل هو درع أساسي لكل أب وأم، وموظف، وطالب، وطبيب، ورجل أعمال لحماية أسرار حياته وأمان أسرته.

---

10. الفرص الكبرى مقابل المخاطر الرقمية

يقف الإنسان في العصر الرقمي أمام مشهد ذي وجهين شديدي التباين:

الوجه المشرق: الفرص غير المسبوقة

- **دمقرطة المعرفة:** أصبح بإمكان أي شاب في قرية نائية أن يتعلم علوم الحاسب والفيزياء والاقتصاد من أقوى جامعات العالم مجاناً وبضغطة زر.
- **التمكين الاقتصادي والعمل المرن:** فتح الإنترنت الباب للعمل عن بُعد، والوصول إلى أسواق العمل العالمية، وبناء مشاريع تجارية ناجحة برؤوس

أموال متناهية الصغر.

- **التواصل الإنساني والتكافل:** إمكانية حشد المجتمعات لنصرة القضايا الإنسانية، وجمع التبرعات، وإنقاذ الأرواح، وتبادل الخبرات عبر الحدود الجغرافية.

الوجه المظلم: التحديات والمخاطر الداهمة

- **تآكل الخصوصية:** تحولت بيانات المستخدمين وسلوكياتهم إلى «النفط الجديد»، وغالباً ما يدفع المستخدم خصوصيته ثمناً للخدمات «المجانية».
- **الجريمة السيبرانية المنظمة:** هجمات الفدية، وسرقة الهويات، والابتزاز الإلكتروني، واختراق الحسابات البنكية.
- **الإدمان الرقمي والتشتت:** خوارزميات مصممة خصيصاً لاستنزاف انتباه الإنسان ووصفحات التواصل التي تغذي مشاعر القلق والمقارنة الاجتماعية.
- **التضليل والتزييف:** انتشار الأخبار الكاذبة والمواد المصنوعة بالذكاء الاصطناعي بسرعة تفوق سرعة انتشار الحقائق بمراحل.

إن دور هذا الكتاب ليس أن يدعوك للانسحاب من التكنولوجيا والعزلة في كهف، بل العكس تماماً: **أن يدخل بك إلى العالم الرقمي وأنت مسلح بالوعي والقوة والتمكن.**

---

11. ملخص الفصل الأول

- العالم الرقمي ليس فضاءً خيالياً، بل بنية تحتية فيزيائية هائلة تعتمد على كابلات الألياف الضوئية الممتدة في قيعان المحيطات ومراكز البيانات العالمية.
- **الإنترنت** هو شبكة الطرق والبنية التحتية، بينما **الويب** هو إحدى السيارات والخدمات التي تسير فوق هذه الشبكة.
- جميع البيانات مهما بلغت درجة تعقيدها تُختزل إلى نبضات كهربائية أو ضوئية تمثل الأصفار والآحاد (Bits).
- تنتقل البيانات عبر الشبكة مقطعة إلى حزم صغيرة (Packets) تقودها الموجهات وتترجم أسماءها خوادم الـ DNS لتصل في أجزاء من الثانية.
- الوعي الرقمي لم يعد خياراً ثانوياً، بل هو الأداة الأساسية لصيانة كرامة الإنسان، وحماية أسرته، وبناء مستقبله المهني في عصر تسوده البيانات.

---

12. اختبر معلوماتك (أسئلة للمراجعة الذاتية)

1. **ما الفارق الجوهري بين شبكة الإنترنت وشبكة الويب العالمية؟ اذكر مثالاً لخدمة تستخدم الإنترنت دون استخدام الويب.**
2. **لماذا تعتمد الحواسيب نظام العد الثنائي (0 و 1) بدلاً من النظام العشري الذي يستخدمه البشر؟**

3. **اشرح باختصار وظيفة نظام أسماء النطاقات (DNS) ولماذا يُشبه بدليل الهاتف.**
4. **ما الفرق الرئيسي بين بروتوكول TCP وبروتوكول UDP في نقل البيانات، ومتى يُفضل استخدام كل منهما؟**

---

13. مصادر للتوسع والقراءة الموصى بها

- **W3C (World Wide Web Consortium):** المعايير التأسيسية لتطور شبكة الويب العالمية عبر الموقع الرسمي: `w3.org` .
- **RFC 793 - Transmission Control Protocol:** الوثيقة المعيارية التاريخية المؤسسة لبروتوكول TCP عبر منظومة `ietf.org` .
- **MDN Web Docs - How the Internet Works:** دليل شبكة مطوري موزيلا التأسيسي للمبتدئين حول ميكانيزمات الاتصال الشبكي.
- **Submarine Cable Map (`submarinecablemap.com`):** خريطة تفاعلية عالمية موثوقة تستعرض مسارات الكابلات البحرية الممتدة في قيعان المحيطات.

الفصل الثاني: كيف تعمل البرامج والتطبيقات؟

> *«البرمجيات ليست سحراً غامضاً، بل هي سلسلة من الأوامر المنطقية الدقيقة التي تجعل قطعة من السيليكون والزجاج تفهم رغبة الإنسان وتنفذها في كسر من الثانية».*

1. افتتاحية: الصندوق الأسود الذي يرافقنا في كل مكان

كل صباح ومساء، تفتح عشرات التطبيقات على هاتفك أو حاسوبك: تطبيق مراسلة، تطبيق بنكي، متصفح، لعبة ثلاثية الأبعاد، منصة فيديو. تنقر بإصبعك، فتتحرك الأزرار، وتصل الرسائل فوراً، وتُخصم الأموال بدقة متناهية.

بالنسبة للأغلبية الساحقة من الناس، يُعد هذا التطبيق "صندوقاً أسود" سحرياً: تضغط هنا، فيحدث ذلك هناك!

ولكن، عندما تتعامل مع الشيء كصندوق سحري غامض، فأنت عاجز عن معرفة:

* أين تذهب أسرارك عند كتابتها؟
* ما الذي يملكه مطورو التطبيق من قدرة للوصول إلى هاتفك؟
* ولماذا تتوقف بعض التطبيقات فجأة أو تخبرك بأن "الخادم لا يستجيب"؟

في هذا الفصل، سنفتح هذا الصندوق الأسود معاً، ونفكك مكوناته قطعة قطعة بلغة بسيطة تبدأ من المعالج الصغير في جهازك، مروراً بقواعد البيانات السحابية، ووصولاً إلى واجهات برمجة التطبيقات (APIs).

---

2. ما هي البرمجيات؟ وطبقة التجريد بين العتاد والإنسان

لنتفق أولاً على قاعدة أساسية: **الحاسوب في حد ذاته كائن "غبي" للغاية لكنه خارق السرعة والدقة.**

المعالج (CPU) لا يفهم ما هو "الحب"، ولا يعرف ما هي "الصورة"، ولا يدرك معنى "حساب بنكي". المعالج ليس سوى مجموعة هائلة من الترانزستورات (مليارات المفاتيح الكهربائية الدقيقة) التي تتبدل بين حالتين فقط: يمر تيار (1) أو لا يمر تيار (0).

مفهوم طبقة التجريد (Abstraction):

كيف ننقل فكرة بشرية معقدة (مثل: "اعرض لي قائمة أصدقائي المتصلين الآن") إلى كهرباء؟

هنا يأتي دور **طبقات التجريد (Layers of Abstraction)**:

1. **العتاد (Hardware):** المعالج، والذاكرة العشوائية (RAM)، ووحدة التخزين، وشاشة اللمس.
2. **لغة الآلة (Machine Code):** الأصفار والآحاد التي يتلقاها المعالج مباشرة.
3. **لغة التجميع ولغات البرمجة (Assembly & High-Level Languages):** كلمات مفهومة للبشر (مثل `if` , `while` , `print`).
4. **نظام التشغيل (Operating System):** المدير العام الذي ينظم استخدام المعالج والذاكرة.
5. **التطبيق (Application):** الواجهة النهائية الرسومية التي يتفاعل معها المستخدم.

بفضل هذه الطبقات، لا يحتاج المبرمج اليوم إلى معرفة شحنة الترانزستور بالمللي فولت ليكتب تطبيقاً؛ فالتكنولوجيا تخفي التعقيد المنخفض لتسمح للإنسان بالتركيز على حل المشكلة.

---

3. أنظمة التشغيل (OS): المدير العام وحارس البوابة

لو لم تكن هناك أنظمة تشغيل، لكان عليك في كل مرة تريد فيها كتابة نص على الشاشة أن تبرمج وحدة معالجة الرسوميات وشاشة اللمس من الصفر!

نظام التشغيل (Operating System - OS) مثل (Android, iOS, Windows, macOS, Linux) هو أهم برنامج في الجهاز على الإطلاق. إنه بمثابة **حكومة الجهاز ومدير موارده**:

- **إدارة المعالج والذاكرة:** يقرر أي تطبيق يعمل الآن في الذاكرة العشوائية (RAM)، وكم ميجابايت يحصل عليها، ويفصل التطبيقات عن بعضها حتى لا يعطل تطبيق فاشل بقية البرامج.
- **إدارة الأمان والبيئة المعزولة (Sandboxing):** يمنع أي تطبيق خبيث أو عادي من التجسس على ذاكرة تطبيق آخر؛ فمثلاً تطبيق اللعبة لا يستطيع قراءة محادثات تطبيق المراسلة لأن نظام التشغيل يضع كل تطبيق في "صندوق رمل" معزول.
- **إدارة العتاد والأذونات:** عندما يطلب تطبيق ما استخدام الكاميرا أو الميكروفون، يتدخل نظام التشغيل فوراً ويسألك: "هل تسمح لهذا التطبيق بالوصول إلى الكاميرا؟". إذا رفضت، يقطع النظام الإشارة المادية عن التطبيق.

---

4. كيف يترجم الحاسوب التعليمات البرمجية؟

عندما يكتب مبرمج كوداً بلغة مثل بايثون (Python) أو جافا سكريبت (JavaScript) أو سي بلس بلس (C++)، كيف يُنفذ هذا الكود؟

هناك طريقتان رئيسيتان:

1. **الترجمة المسبقة (Compilation):**

يتم فحص الكود بالكامل مرة واحدة بواسطة برنامج يُسمى **المترجم (Compiler)** وتحويله مسبقاً إلى ملف تنفيذي بلغة الآلة الصرفة (مثل

ملفات `.exe` في ويندوز). هذه البرامج تكون فائقة السرعة عند التشغيل (مثل لغات C و C++ و Rust و Go).

2. **التفسير الفوري (Interpretation):**
يتم قراءة الكود سطراً بسطر بواسطة برنامج وسيط يُسمى **المُفسر (Interpreter)** أثناء تشغيل البرنامج وتنفيذه فوراً. هذه البرامج أسهل في التطوير والتعلم (مثل لغة Python).

---

5. تشريح أنواع التطبيقات: الهواتف، الويب، وسطح المكتب

ليست كل البرامج مبنية بنفس الأسلوب؛ فهناك بيئات مختلفة تفرض هندسة متباينة:

تطبيقات الهاتف الذكي (Mobile Apps)

- **التطبيقات الأصيلة (Native Apps):** كُتبت خصيصاً لنظام معين (بـ Kotlin/Java لنظام Android، أو بـ Swift لنظام iOS). تتميز بالسرعة الفائقة والوصول الكامل لعتاد الهاتف (مستشعرات الحركة، الكاميرا بدقة كاملة).
- **التطبيقات الهجينة ومتعددة المنصات (Cross-Platform Apps):** تُكتب مرة واحدة (باستخدام أدوات مثل Flutter أو React Native) لتعمل على Android و iOS معاً، مما يوفر وقت التطوير وتكلفة الصيانة.

تطبيقات الويب (Web Applications)

تطبيقات لا تحتاج إلى تثبيتها من متجر؛ بل تعمل داخل متصفح الإنترنت مباشرة (مثل محرر مستندات جوجل، أو موقع فيسبوك). تنقسم بنيتها إلى شقين:

* **الواجهة الأمامية (Frontend):** كل ما تراه عيناك وتلمسه أصابعك في المتصفح، وتُبنى بتقنيات (HTML للهيكل، CSS للتصميم والألوان، وJavaScript للحركات والتفاعل).
* **الواجهة الخلفية (Backend):** العقل المدبر الذي يعمل على خوادم بعيدة، يعالج المنطق الحسابي، يتحقق من صلاحياتك، ويتعامل مع قواعد البيانات.

التطبيقات المكتبية (Desktop Applications)

تطبيقات تُثبت محلياً على الحواسيب الشخصية وتستفيد من كامل عتاد الحاسوب والقرص الصلب والبطاقة الرسومية القوية (مثل برامج المونتاج Photoshop وPremiere، وألعاب الفيديو الضخمة).

---

6. قواعد البيانات: مستودعات الذاكرة الرقمية

لو كان التطبيق إنساناً، لكان المعالج هو «عقله المفكر»، والواجهة هي «ملامحه وصوته»، ولكانت **قاعدة البيانات (Database)** هي «ذاكرته الدائمة».

بدون قاعدة البيانات، ينسى التطبيق كل شيء بمجرد إغلاقه!

تنقسم قواعد البيانات الحديثة أساساً إلى نوعين:

1. **قواعد البيانات العلائقية (Relational / SQL):**

تخزن البيانات في جداول منتظمة ومحكمة تشبه جداول Excel لكن بقدرات عملاقة، تربط بينها علاقات صارمة (مثل PostgreSQL و MySQL). ممتازة للبيانات المالية والحسابات التي تتطلب دقة متناهية لا تحتمل الخطأ.

2. **قواعد البيانات غير العلائقية (NoSQL):**

تخزن البيانات كوثائق مرنة ومفتوحة (مثل MongoDB)، ممتازة للشبكات الاجتماعية والبيانات الضخمة المتغيرة السرعة.

---

7. الخوادم السحابية ونماذج الحوسبة (IaaS, PaaS, SaaS)

في السابق، كانت أي شركة تريد إطلاق تطبيق تضطر لشراء حواسيب ضخمة ووضعها في غرفة مبردة واستئجار خطوط إنترنت باهظة وفريق صيانة. أما اليوم، فالحوسبة السحابية (Cloud Computing) أتاحت استئجار هذه الموارد بضغطة زر.

تنقسم الخدمات السحابية عالمياً إلى ثلاثة نماذج مشهورة:

النموذج	الاسم الكامل	المعنى البسيط	مثال واقعي
IaaS	البنية التحتية كخدمة	تستأجر عتاداً افتراضياً خاماً (معالج، رام، قرص) وتثبت عليه ما تشاء	AWS EC2, Google Cloud Compute
PaaS	المنصة كخدمة	تستأجر بيئة جاهزة لتشغيل كودك دون أن تقلق بشأن تحديث نظام التشغيل	Render, Heroku, Firebase
SaaS	البرنامج كخدمة	تستخدم التطبيق النهائي مباشرة دون أدنى علاقة بما يدور خلفه	Gmail, Google Drive, Office 365

---

8. واجهات برمجة التطبيقات (APIs): جسور التفاهم الرقمي

إذا أردت أن تفهم سحر العالم الرقمي الحديث، فعليك أن تفهم ما هو الـ **API (Application Programming Interface)**.

تشبيه المطعم الشهير:

تخيل أنك في مطعم:

* أنت (العميل) تجلس على الطاولة، وتريد طبقاً معيناً.

* المطبخ (الخادم وقاعدة البيانات) هو المكان الذي تُحضر فيه الوجبات ولديه

كل المكونات السرية.
* لا يُسمح لك بالدخول إلى المطبخ بنفسك والعبث بالأواني!
* من الذي يربط بينك وبين المطبخ؟ **«النادل» (الـ API)**.
يأتي إليك، يأخذ طلبك المنظم، يوصله إلى المطبخ، ثم يعود إليك بالطبق المطلوب!

مثال واقعي:

عندما تحجز تذكرة طيران عبر تطبيق مقارنة أسعار، كيف يعرف التطبيق مواعيد وأسعار 50 شركة طيران مختلفة في ثانية واحدة؟
هل يدخل قاعدة بيانات كل شركة طيران؟ مستحيل!
بدلاً من ذلك، توفر كل شركة طيران **API آمن** يستقبل سؤال التطبيق: "ما الرحلات المتاحة من دبي إلى لندن يوم الأحد؟"، فيرد الـ API بالبيانات الدقيقة المشفرة فوراً.

الـ APIs هي الشرايين التي جعلت التطبيقات تتحدث مع بعضها وتتكامل في منظومة عالمية واحدة.

---

9. تفكيك رحلة واقعية: ماذا يحدث عند تسجيل الدخول في تطبيق؟

لندمج ما تعلمناه في سيناريو واحد دقيق: أنت تفتح تطبيقك البنكي، وتكتب اسم المستخدم وكلمة المرور، ثم تضغط "تسجيل الدخول". ماذا يجري خلف

الكواليس؟

```
[هاتفك الذكي] ---> (طلب مشفر بـ HTTPS) ---> [بوابة الـ API]
                                                    |
                                                    v
                                              [خادم التوثيق]
                                                    |
                                                    v
                                              [قاعدة البيانات المشفرة]
```

1. **التحقق المحلي (Client-side Validation):** يتأكد التطبيق في هاتفك من أنك لم تترك الحقول فارغة وأن صيغة البريد صحيحة.
2. **التشفير عبر النقل (TLS/HTTPS):** تُشفر كلمة المرور في حزمة بيانات آمنة قبل خروجها من هاتفك حتى لا يقرأها أي شخص يتنصت على شبكة الواي فاي.
3. **الاستقبال لدى بوابة الـ API:** يستلم خادم البنك الطلب ويوجهه إلى "خدمة التوثيق والهوية" (Auth Service).
4. **التجزئة والمطابقة (Hashing):** البنك الآمن **لا يخزن كلمة مرورك كنص صريح أبداً**؛ بل يمرر ما كتبته عبر خوارزمية رياضية معقدة لإنتاج "بصمة رقمية" ومقارنتها بالبصمة المحفوظة في قاعدة البيانات (سنشرح هذا بالتفصيل في الفصل السادس).
5. **توليد رمز الجلسة (Token Generation):** إذا تطابقت البصمة، ينشئ الخادم رمزاً مشفراً مؤقتاً يُسمى (Access Token) مثل بطاقة دخول الفندق

الرقمية، ويرسله إلى هاتفك.

6. **فتح الواجهة:** يحفظ هاتفك هذا الرمز ويستخدمه مع كل طلب قادم ليثبت للخادم أنك أنت، وتفتح شاشتك الرئيسية لترى رصيدك المالي بسلام!

---

10. دورة حياة تطوير البرمجيات (SDLC)

بناء التطبيقات ليس مجرد جلوس مبرمج أمام شاشة سوداء ليكتب أكواداً عشوائية طوال الليل كما تصور أفلام هوليوود!
إن هندسة البرمجيات تتبع دورة حياة منهجية منضبطة تُعرف بـ **SDLC (Software Development Life Cycle)**:

1. **التحليل وجمع المتطلبات (Requirements):** ما المشكلة التي نريد حلها؟ ومن هو المستخدم المستهدف؟
2. **التصميم (UI/UX Design):** رسم الواجهات وتجربة المستخدم قبل كتابة أي سطر كود.
3. **التطوير البرمجي (Implementation / Coding):** بناء الواجهات الأمامية والأنظمة الخلفية وقواعد البيانات.
4. **الاختبار وضمان الجودة (Testing & QA):** فحص التطبيق لكشف الأخطاء والثغرات والتأكد من تحمله للضغط.
5. **النشر والإطلاق (Deployment):** رفع التطبيق إلى المتاجر والخوادم السحابية للمستخدمين.

6. **الصيانة والتحديث المستمر (Maintenance & Updates):** سد الثغرات المكتشفة، وتحسين الأداء، وإضافة ميزات جديدة استجابة لآراء المستخدمين.

---

11. للقارئ المتخصص: معمارية RESTful APIs وبروتوكول HTTP

🔬 للقارئ المتخصص (Technical Deep Dive) تعتبر معمارية **REST (Representational State Transfer)** النمط الأكثر شيوعاً لتصميم واجهات الويب البرمجية عبر بروتوكول HTTP. #### المبادئ الستة الأساسية لمعمارية REST: 1. **فصل العميل عن الخادم (Client-Server Separation):** استقلالية منطق الواجهات عن منطق معالجة البيانات وتخزينها. 2. **انعدام الحالة (Statelessness):** كل طلب HTTP يرسله العميل يجب أن يحتوي على جميع المعلومات اللازمة لفهمه ومعالجته بصورة مستقلة، دون أن يعتمد الخادم على تخزين حالة الجلسة في ذاكرته. 3. **قابلية التخزين المؤقت (Cacheability):** يجب أن تحدد الاستجابات صراحة ما إذا كانت قابلة للتخزين المؤقت أم لا لتقليل استهلاك الشبكة. 4. **الواجهة الموحدة (Uniform Interface):** استخدام عناوين URIs قياسية لتمثيل الموارد (Resources). 5. **النظام الطبقي (Layered System):** يمكن للطلب أن يمر عبر طبقات وسيطة (Load Balancers, API Gateways, Firewalls) دون علم العميل. 6.

الكود عند الطلب (Code on Demand - اختياري): إمكانية إرسال سكربتات لتنفيذها على العميل. #### أفعال HTTP القياسية (HTTP Verbs): * `GET`: لاسترجاع مورد (آمن وغير معدل للحالة Idempotent). * `POST`: لإنشاء مورد جديد داخل النظام. * `PUT`: لتحديث كامل لمورد موجود. * `PATCH`: لتحديث جزئي لمورد موجود. * `DELETE`: لحذف مورد محدد. #### فئات رموز الحالة (HTTP Status Codes): * `2xx (Success)`: أشهرها `200 OK` للنجاح العادي، و`201 Created` لنجاح الإنشاء. * `3xx (Redirection)`: أشهرها `301 Moved Permanently` و`304 Not Modified`. * `4xx (Client Error)`: أشهرها `400 Bad Request` للبيانات الخاطئة، و`401 Unauthorized` لغياب التوثيق، و`403 Forbidden` لمنع الوصول، و`404 Not Found` لعدم وجود المورد. * `5xx (Server Error)`: أشهرها `500 Internal Server Error` و`502 Bad Gateway` و`503 Service Unavailable`.

---

12. ملخص الفصل الثاني

- البرمجيات ليست سحراً، بل طبقات تجريد منطقية تبدأ من مفاتيح السيليكون في المعالج وحتى الواجهات الرسومية البراقة.
- نظام التشغيل هو المدير وحارس البوابة؛ يفصل بين البرامج، ويمنح الموارد، ويحمي الأجهزة من استغلال التطبيقات.

- تنقسم البرمجيات إلى تطبيقات هواتف أصيلة وهجينة، وتطبيقات ويب (واجهة أمامية وخلفية)، وتطبيقات مكتبية.
- قواعد البيانات هي المستودعات الدائمة للمعلومات، والحوسبة السحابية أتاحت استئجار البنية التحتية بنماذج (IaaS, PaaS, SaaS).
- واجهات برمجة التطبيقات (APIs) هي جسور التفاهم الآمنة بين النظم الرقمية المختلفة وبدونها لا يمكن للتطبيقات التكامل.
- بناء البرمجيات عملية هندسية تتبع دورة حياة دقيقة تبدأ بالتخطيط وتنتهي بالصيانة والتحديث المستمر.

---

13. اختبر معلوماتك (أسئلة للمراجعة الذاتية)

1. **ما هي وظيفة «نظام التشغيل» في حماية أمن التطبيقات وعزلها عن بعضها (Sandboxing)؟**
2. **اشرح مفهوم «الـ API» مستخدماً تشبيه المطعم أو مثالاً واقعياً من اختيارك.**
3. **ما الفرق بين الواجهة الأمامية (Frontend) والواجهة الخلفية (Backend) في تطبيقات الويب؟**
4. **لماذا يُعد تسجيل الدخول الآمن عملية متعددة الخطوات تتطلب التشفير والتجزئة بدلاً من إرسال كلمة المرور بنص صريح؟**

---

14. مصادر للتوسع والقراءة الموصى بها

- **MDN Web Docs - Client-Server Overview:** الدليل المعياري لتشريح الاتصال بين الواجهات الأمامية والخوادم عبر شبكة موزيلا.
- **OpenAPI Specification (OAS):** المواصفة العالمية القياسية لتوثيق وتصميم واجهات برمجة التطبيقات RESTful APIs عبر `openapis.org` .
- **Postman API Architecture Guide:** مقالات تعليمية موثوقة حول تصميم وتكامل واجهات برمجة التطبيقات واختبارها.

الفصل الثالث: البرمجة وفن التفكير المنطقي

> *«البرمجة لا تعلمك فقط كيف تتحدث إلى الآلة، بل تعلمك قبل ذلك كيف ترتب أفكارك، وكيف تفكك المشكلات الكبيرة إلى خطوات صغيرة تستطيع حلها بهدوء».*

1. افتتاحية: هل البرمجة للمبرمجين فقط؟

هناك خرافة شائعة تقول: *"البرمجة مهارة مقتصرة على العباقرة وخبراء الرياضيات الذين يجلسون أمام شاشات زرقاء في غرف مظلمة!"*.

الحقيقة على النقيض من ذلك تماماً: **البرمجة هي فن التفكير المنظم**.

عندما تتعلم البرمجة، أنت لا تتعلم حفظ مصطلحات إنجليزية غريبة، بل تكتسب **"عقلية جديدة"** في التعامل مع الحياة:

* تتعلم ألا تصاب بالذعر أمام المشاكل الضخمة؛ بل تقسمها إلى أجزاء صغيرة يمكن إدارتها.
* تتعلم البحث عن السبب الجذري للخلل بدلاً من الاكتفاء بالشكوى من النتيجة.
* تتعلم الدقة؛ فالأشياء إما أن تكون صحيحة ومحددة أو أنها لا تعمل!

في هذا الفصل، سنخطو معاً أولى خطواتنا في عالم البرمجة باستخدام لغة **بايثون (Python)**—أشهر وأجمل لغات العالم اليوم وأقربها للغة الإنسانية الطبيعية. سنفهم القواعد الجوهرية التي تبنى عليها كل برامج الأرض، ونبني معاً في نهاية الفصل برنامجاً عملياً نافعاً يخدم المجتمع.

---

2. ما هي البرمجة؟ وما هي الخوارزمية؟

البرمجة (Programming) ببساطة هي: إعطاء مجموعة من التعليمات المرتبة منطقياً للحاسوب ليقوم بمهمة محددة.

والقلب النابض لأي برنامج في العالم هو **«الخوارزمية» (Algorithm)**.

ما أصل الخوارزمية؟

سُميت الخوارزمية بهذا الاسم نسبة إلى العالم المسلم العبقري **أبي عبد الله محمد بن موسى الخوارزمي** (عاش في القرن التاسع الميلادي)، مؤسس علم الجبر وأول من وضع أسس الخطوات المنطقية المتسلسلة لحل المعادلات الحسابية.

تشبيه وصفة الطهي:

فكر في الخوارزمية كوصفة دقيقة لإعداد كعكة:
1. أحضر ثلاثة أكواب من الطحين وملعقة سكر.
2. اخفق البيض جيداً لمدة دقيقتين.
3. امزج الخليط في وعاء واسع.
4. ضعه في الفرن عند درجة حرارة 180 مئوية لمدة نصف ساعة.

إذا عكست خطوة رقم 4 قبل خطوة رقم 2، فلن تحصل على كعكة أبداً! الحاسوب بالضبط كالمطبخ: ينفذ تعليماتك بحرفية شديدة وبنفس الترتيب الذي كتبته له. إذا كانت خوارزميتك واضحة، نجح البرنامج؛ وإذا كان فيها خلل، أخطأ الحاسوب.

---

3. لغة بايثون (Python): لماذا هي بوابتنا المثالية؟

تتميز لغة بايثون بأنها صُممت من الأساس لتكون **مقروءة، وبسيطة، وأنيقة**. فبينما تتطلب بعض اللغات كتابة 10 أسطر معقدة لطباعة جملة، تفعلها بايثون في سطر واحد بديهي:

```
print("مرحباً بك في عالم الوعي الرقمي!")
```

تُستخدم بايثون اليوم في كبرى الشركات العالمية والجامعات: في تطوير مواقع الويب، في تحليل البيانات، في تطبيقات الأمن السيبراني الدفاعي، وفي

أبحاث الذكاء الاصطناعي الأكثر تطوراً.

---

4. اللبنات التأسيسية للبرمجة (The Core Building Blocks)

مهما بلغت درجة تعقيد أي برنامج في العالم—من لعبة ثلاثية الأبعاد إلى نظام توجيه الصواريخ—فإنه مبني في جوهره من أربعة عناصر رئيسية:

أ) المتغيرات وأنواع البيانات (Variables & Data Types)

المتغير هو بمثابة "صندوق تخزين يحمل اسماً" داخل ذاكرة الحاسوب المؤقتة (RAM)، تضع فيه قيمة لاستخدامها لاحقاً:

```
# أمثلة على المتغيرات في بايثون
user_name = "نص" # جنكيز (String / str)
user_age = 25 # عدد صحيح (Integer / int)
account_balance = 1500.75 # عدد عشري (Float / float)
is_authenticated = True # قيمة منطقية (Boolean: True أو False)
```

ب) الشروط واتخاذ القرار (Conditionals: if / else)

البرنامج الذكي هو الذي يعرف كيف يتخذ قرارات وفق المعطيات:

```
 password_attempts = 3

if password_attempts >= 3:
    print("تم قفل الحساب مؤقتاً لحمايتك!")
else:
    print("يمكنك محاولة كتابة كلمة المرور مجدداً.")
```

ج) التكرار الذكي والحلقات (Loops: for / while)

إذا طلبت من إنسان كتابة جملة 1000 مرة، فسيشعر بالملل والإرهاق بعد المرة العاشرة. أما الحاسوب، فيكرر المهمة ملايين المرات في ثانية واحدة دون أدنى تذمر:

```
 # فحص قائمة الرسائل الواردة لكشف الروابط المشبوهة
messages = ["أهلاً بك", "مبروك ربحت مليون دولار! انقر هنا", ""اجتماع العمل غداً"]

for msg in messages:
    if "مبروك ربحت" in msg:
        print(f"تحذير: رسالة احتيال محتملة -> {msg}")
```

د) الدوال وإعادة الاستخدام (Functions & Modularity)

بدلاً من إعادة كتابة نفس الكود 50 مرة في أماكن مختلفة، نجمع الخطوات داخل **«دالة» (Function)** وننادي باسمها كلما احتجنا إليها:

```
 def calculate_tax(amount):
    """دالة تحسب الضريبة بنسبة 15%"""
```

```
    return amount * 0.15

# استدعاء الدالة بسهولة
final_tax = calculate_tax(200)
print(f"قيمة الضريبة هي: {final_tax} ﷼")
```

---

5. معالجة الأخطاء والتعامل مع الاستثناءات (Error Handling)

المبرمج المبتدئ يفترض دائماً أن المستخدم سيكتب مدخلات مثالية ولن تقع أية مشكلة في الشبكة. أما المبرمج المحترف، فيتوقع الخطأ قبل وقوعه ويحصن برنامجه ضده!

ماذا يحدث لو طلب برنامجك من المستخدم إدخال عمره، فكتب كلمة "خمسة وعشرون" بالحروف بدلاً من الأرقام؟
بدون حماية، سينهار البرنامج ويتوقف فوراً (Crashes).

لحماية البرنامج، نستخدم ما يُسمى **كتلة الحماية (try / except)**:

```
 try:
    age_input = input("من فضلك اكتب عمرك بالأرقام: ")
    age = int(age_input) # محاولة تحويل النص إلى رقم
    print(f"عمرك المسجل هو: {age} عاماً")
except ValueError:
    print("تنبيه لطيف: يرجى إدخال أرقام صحيحة فقط (مثال: 25) وليس كلمات.")
```

بهذه الطريقة البسيطة، يظل البرنامج ثابتاً ورصيناً ويرشد المستخدم باحترام دون أن ينهار.

---

6. قواعد الكود النظيف (Clean Code)

الكود البرمجي لا يُكتب فقط لتقرأه الآلة وتنفذه، بل يُكتب في المقام الأول **ليقرأه إنسان آخر يفهمه ويطوره**.

لذلك، يجب أن تتبع هذه القواعد الذهبية:

1. **اختر أسماء ذات معنى للمتغيرات:** استخدم `user_email` بدلاً من `x` أو `a1`.
2. **اكتب دوال صغيرة ومركزة:** كل دالة يجب أن تؤدي مهمة واحدة واضحة ومحددة.
3. **أضف التعليقات التوضيحية المفيدة:** اشرح *«لماذا»* قمت بهذا الحل، وليس مجرد إعادة سرد ما يفعله الكود بديهياً.
4. **التنسيق والمسافات البادئة:** لغة بايثون تعتمد على المسافات البادئة (Indentation) لترتيب الكتل المنطقية؛ حافظ على تناسقها.

---

7. مشروع الفصل التطبيقي: برنامج إدارة الميزانية والتبرع المجتمعي

تطبيقاً عملياً لرسالة هذا الكتاب في ربط البرمجة بخدمة المجتمع، سنقوم الآن بكتابة وتشغيل برنامج متكامل بلغة بايثون لإدارة المصروفات الشهرية وحساب نسبة مخصصة للتبرع ودعم المحتاجين.

هذا البرنامج آمن تماماً، ويعمل محلياً، ويجمع بين: المتغيرات، الدوال، الحلقات، الشروط، ومعالجة الأخطاء.

```
 # ============================================================
=======
# مشروع الفصل الثالث: مدير الميزانية والمساهمة المجتمعية
# تأليف وفكرة: كتاب الوعي الرقمي - جنكيز
# ============================================================
======

def calculate_budget(income, expenses, donation_rate=0.0
5):
    """
    دالة لحساب صافي الميزانية وتخصيص نسبة للتبرع المجتمعي
    :param income: إجمالي الدخل الشهري
    :param expenses: قائمة بقيم المصروفات
    :param donation_rate: نسبة العطاء المجتمعي المقترحة (ا
(%5 لافتراضي
    :return: قاموس يحتوي على الملخص المالي
    """
    total_expenses = sum(expenses)
    remaining_balance = income - total_expenses

    # حساب نسبة التبرع من الرصيد المتبقي إذا كان إيجابياً
    if remaining_balance > 0:
```

```
        suggested_donation = remaining_balance * donation_
rate
        net_savings = remaining_balance - suggested_donati
on
    else:
        suggested_donation = 0.0
        net_savings = remaining_balance

    return {
        "income": income,
        "total_expenses": total_expenses,
        "remaining_balance": remaining_balance,
        "donation": suggested_donation,
        "net_savings": net_savings
    }

def display_report(summary):
    """طباعة تقرير مالي ومجتمعي منسق للمستخدم"""
    print("\n" + "=" * 48)
    print("      📊 تقرير ميزانيتك ووعيك المالي والمجتمعي
")
    print("=" * 48)
    print(f"💰 إجمالي الدخل الشهري      : {summary['incom
e']:.2f}")
    print(f"📉 إجمالي المصروفات        : {summary['total_e
xpenses']:.2f}")
    print(f"💵 الرصيد المتبقي           : {summary['remain
ing_balance']:.2f}")
    print("-" * 48)

    if summary['remaining_balance'] > 0:
        print(f"🤝 حصة العطاء المجتمعي (%5)  : {summary['d
onation']:.2f}")
        print(f"🌱 صافي الادخار الشخصي      : {summary['ne
t_savings']:.2f}")
        print("💡 رسالة: «المعرفة قوة، لكن قيمتها الحقيقية
تتجلّى في نفع الآخرين بها»")
```

```
    else:
        print("⚠️ تنبيه: مصروفاتك تجاوزت دخلك! راجع بنود الإنفاق بعناية.")
    print("=" * 48 + "\n")

# تجربة تشغيل البرنامج بأرقام واقعية
if __name__ == "__main__":
    monthly_income = 5000.0  # دخل شهري افتراضي
    monthly_expenses = [1200.0, 450.0, 300.0, 650.0]  # إيجار، طعام، إنترنت، فواتير

    report = calculate_budget(monthly_income, monthly_expenses)
    display_report(report)
```

تفكيك الكود البرمجي:

- `def calculate_budget(...)` : تعريف دالة مرنة تقبل الدخل وقائمة المصروفات وتحسب العمليات بدقة متناهية.
- `sum(expenses)` : استخدام الدوال المدمجة في بايثون لجمع كافة عناصر القائمة في سطر واحد.
- **الشرط المنطقي** `if remaining_balance > 0` : التأكد من أن التبرع لا يُقترح إلا إذا كان هناك فائض مالي إيجابي يحمي كرامة المستخدم وميزانيته الأساسية.
- **التنسيق** `:.2f` : أمر بايثون بعرض الأرقام العشرية بخانتين فقط بعد الفاصلة، لتبدو الأرقام كعملات مالية احترافية.

---

8. للقارئ المتخصص: التعقيد الخوارزمي ونظرية Big O

🔬 للقارئ المتخصص (Technical Deep Dive) لا يقاس الكود البرمجي الاحترافي بمجرد أنه "يعمل"، بل يقاس أيضاً بـ **كفاءته واستهلاكه لموارد النظام (الزمن والذاكرة)** مع تضخم حجم المدخلات N. يُعبر علماء الحاسوب عن هذا المفهوم بـ **ترميز Big O (Big O Notation)**: 1. **التعقيد الثابت $O(1)$:** العملية تستغرق نفس الزمن دائماً بصرف النظر عن حجم البيانات (مثال: استخراج عنصر من مصفوفة عبر فهرسه المباشر `array[0]`). 2. **التعقيد اللوغاريتمي $O(\log N)$:** كفاءة فائقة؛ يتضاعف حجم البيانات ملايين المرات بينما يزيد زمن المعالجة بنسب طفيفة (أشهر مثال: **خوارزمية البحث الثنائي Binary Search** في قائمة مرتبة). 3. **التعقيد الخطي $O(N)$:** يزيد الزمن طردياً وبصورة مباشرة مع زيادة البيانات (مثال: حلقة تمر على كل عناصر قائمة غير مرتبة للبحث عن اسم). 4. **التعقيد التربيعي $O(N^2)$:** حلقات متداخلة (Nested Loops) حيث تفحص كل عنصر مقابل كل العناصر الأخرى؛ يعتبر كارثياً عند التعامل مع قواعد بيانات تضم ملايين السجلات ويجب تجنبه في الأنظمة الحساسة. #### نصيحة هندسية دفاعية: في برمجيات الأمن وتطبيقات الويب، يمكن لخوارزمية غير كفؤة ذات تعقيد $O(N^2)$ أن تفتح الباب لما يُعرف بـ **هجمات حجب الخدمة الخوارزمية (Algorithmic Denial of Service)**، حيث يرسل المهاجم مدخلات مصممة خصيصاً لتستهلك 100% من قدرة المعالج وتعطل الخادم!

9. ملخص الفصل الثالث

- البرمجة ليست مجرد كتابة تعليمات لآلة، بل هي منهجية لتنظيم التفكير وحل المشكلات وتفكيك المعضلات إلى خطوات واضحة.
- الخوارزمية هي وصفة الحل المنطقي الدقيق، وسُميت تكريماً للعالم المسلم محمد بن موسى الخوارزمي.
- لغة بايثون هي الأداة المثالية لبدء تعلم البرمجة نظراً لبساطتها وقوتها وانتشارها في كافة المجالات التقنية الحديثة.
- اللبنات الأربع لأي برنامج: المتغيرات، الجمل الشرطية، الحلقات التكرارية، والدوال القابلة لإعادة الاستخدام.
- معالجة الأخطاء (Error Handling) عبر كتل `try/except` تميز المبرمج المحترف وتمنع انهيار البرامج أمام المستخدمين.
- كتابة الكود النظيف والمقروء مسؤولية أخلاقية تضمن استدامة الأنظمة وتيسر تعاون فرق العمل لخدمة المجتمع.

---

10. اختبر معلوماتك (أسئلة للمراجعة الذاتية)

1. **ما هي الخوارزمية؟ ولماذا لا يستطيع الحاسوب حل المشكلة إذا كانت خطوات الخوارزمية غير مرتبة منطقياً؟**

2. **ما الفارق بين المتغير من النوع العددي الصحيح (int) والمتغير المنطقي (boolean)؟ اذكر مثالاً لكل منهما.**
3. **كيف تحمي كتلة `try/except` البرامج من الانهيار المفاجئ عند إدخال بيانات غير متوقعة؟**
4. **في نظرية Big O، لماذا يفضل مهندسو البرمجيات خوارزمية ذات تعقيد $O(\log N)$ على خوارزمية ذات تعقيد $O(N^2)$ في قواعد البيانات الكبيرة؟**

---

11. مصادر للتوسع والقراءة الموصى بها

- **Python Official Documentation (`docs.python.org`):** الدليل الرسمي المعتمد للغة بايثون وتوثيق مكتبتها المعيارية.
- **Automate the Boring Stuff with Python (Al Sweigart):** كتاب مرجعي عملي للمبتدئين لتوظيف بايثون في حل المهام اليومية النافعة.
- **Clean Code: A Handbook of Agile Software Craftsmanship (Robert C. Martin):** المرجع العالمي الأهم في كتابة الشيفرات البرمجية النظيفة والمستدامة.

الفصل الرابع: الأمن السيبراني وفهم المخاطر الرقمية

> *«الأمن السيبراني لا يتعلق بحماية الآلات والأسلاك، بل بحماية كرامة الإنسان، وأسراره، ومستقبله، واستقرار مجتمعه في فضاء بلا حدود جغرافية».*

1. افتتاحية: وهم الحصن المنيع

يعتقد كثير من الناس أن الأمن السيبراني مسألة تخص الحكومات والبنوك الكبرى والجيوش فقط، وأن "المستخدم البسيط" ليس لديه ما يخسره أو ما يثير اهتمام القراصنة.

وهذا هو الخطأ القاتل الأول!

في العالم الرقمي، أنت لست مستهدفاً فقط لأنك تملك ملايين الدولارات في حسابك، بل قد تُستهدف لأنك:

* تملك **هاتفاً متصلاً بالإنترنت** يمكن تحويله إلى جهاز ضمن شبكة روبوتية (Botnet) لمهاجمة مستشفى أو تعطيل محطة كهرباء دون علمك!
* تملك **حساب بريد إلكتروني موثوقاً** يمكن استغلاله لخداع أصدقائك أو أفراد عائلتك ومطالبتهم بأموال باسمك.
* تملك **بيانات وذكريات شخصية وصوراً أسرية** يمكن لبرمجية خبيثة تشفيرها وابتزازك لدفع فدية مالية لاستعادتها.

الأمن السيبراني في جوهره ليس أداة للسيطرة أو الهجوم، بل هو **علم وفن الدفاع** عن الفضاء الحيوي الذي نعيش فيه جميعاً.

---

2. ما هو الأمن السيبراني الدفاعي؟

الأمن السيبراني (Cybersecurity): هو مجموعة الممارسات والتقنيات والسياسات والإجراءات الهندسية المصممة لحماية الأنظمة، والشبكات، والبرمجيات، والبيانات من الوصول غير المصرح به، أو التعديل، أو التخريب، أو الاستغلال.

لماذا نركز في هذا الكتاب على "الأمن الدفاعي"؟

لأن نشر المعرفة الهجومية وأساليب الاعتداء والاختراق غير المصرح بها هو عمل ضار بالمجتمع ومخالف للقوانين والأخلاق. أما **الأمن الدفاعي**

(Defensive Security)، فهو الذي يبني الدروع، ويسد الثغرات، ويعلم الأسر والشركات كيف تحمي كرامتها وخصوصيتها.

إن معرفة كيف يفكر المهاجم لا تكون إلا لغرض واحد نبيل: **أن تعرف أين تضع خط دفاعك لتحمي الأبرياء**.

---

3. معادلة الخطر: الفرق بين التهديد، الثغرة، والمخاطرة

لكي تتحدث وتفكر كخبير أمني واعي، يجب أن تضبط الفروق بين ثلاثة مصطلحات يخلط بينها الكثيرون:

1. **التهديد (Threat):**
هو أي خطر خارجي محتمل يملك القدرة على إلحاق الأذى. (مثال: محتال إلكتروني، برمجية خبيثة، فيضان مائي قد يغرق خوادم الشركة).
التهديد كائن موجود في البيئة لا تملك السيطرة على وجوده.

2. **الثغرة الأمنية (Vulnerability):**
هي نقطة ضعف أو خطأ في تصميم البرنامج أو إعدادات النظام أو سلوك المستخدم تسمح للتهديد بالنفاذ. (مثال: ترك باب المنزل دون قفل، أو استخدام كلمة مرور من أربعة أرقام "1234").
الثغرة هي مسؤوليتك أنت، وتستطيع إغلاقها.

3. **المخاطرة (Risk):**
هي الاحتمالية الرياضية والعملية لحدوث ضرر ناتج عن استغلال **تهديد**

معين لـ **ثغرة** موجودة، وحجم الأثر الناتج عن ذلك.

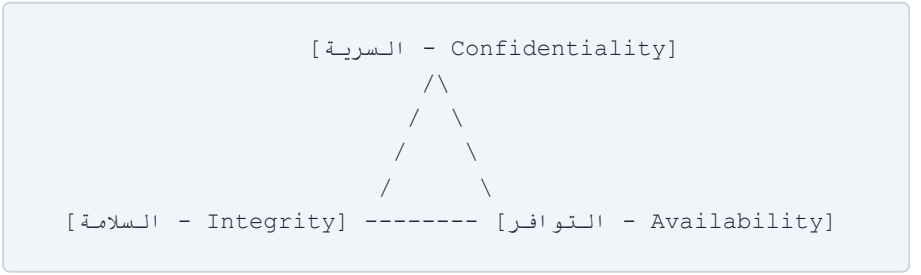
$$\text{المخاطرة (Risk)} = \text{التهديد (Threat)} \times \text{الثغرة (Vulnerability)} \times \text{الأثر (Impact)}$$

إذا وُجد تهديد كبير (لصوص في المنطقة) لكن منزلك محصن تماماً بأقفال ذكية وكاميرات (الثغرة صفر)، فإن مستوى المخاطرة يقترب من الصفر!
لذلك، فإن وظيفة الوعي الرقمي هي **سد الثغرات** حتى تبطل مفعول التهديدات المحيطة بك.

---

4. ثالوث الأمان المعلوماتي الكلاسيكي (CIA Triad)

على مدار أكثر من أربعة عقود، اعتمد مهندسو الأمن العالميون نموذجاً قياسياً يمثل الأضلاع الثلاثة الأساسية لحماية أي معلومة أو نظام، يُعرف عالمياً بـ **ثالوث CIA**:



أ) السرية (Confidentiality): "لا يقرأها إلا من يحق له"

ضمان أن البيانات لا يطلع عليها إلا الأشخاص أو الأنظمة المصرح لهم فقط.
أدوات تحقيقها: التشفير، كلمات المرور القوية، الصلاحيات المحدودة.

ب) السلامة والتكامل (Integrity): "لا يُعدل عليها إلا بحق"

ضمان أن البيانات تظل دقيقة، وصحيحة، وكاملة، ولم يتم التلاعب بها أو تغييرها أثناء التخزين أو النقل.
مثال: عندما تحول 100 ريال لصديقك، يجب ألا يستطيع مهاجم في الطريق تغيير المبلغ إلى 1000 ريال!
أدوات تحقيقها: دوال التجزئة الرياضية (Hashing)، والتوقيعات الرقمية (Digital Signatures).

ج) التوافر والجاهزية (Availability): "تجدها عندما تحتاجها"

ضمان أن النظام والخدمات والبيانات تظل متاحة وجاهزة للعمل كلما طلبها المستخدم المصرح له.
مثال: تطبيق الطوارئ أو البنك يجب ألا يتعطل عندما يحتاجه المريض أو العميل في منتصف الليل.
أدوات تحقيقها: الخوادم الاحتياطية (Redundancy)، النسخ الاحتياطي المنتظم، والحماية ضد هجمات حجب الخدمة (DDoS).

---

5. الهندسة الاجتماعية (Social Engineering): اختراق العقل البشري

يقول خبير الأمن العالمي الشهير كيفن ميتنيك في كتابه *فن الخداع*:

> *«إن أصعب جدران الحماية الرقمية وأقوى خوارزميات التشفير الرياضية تصبح عديمة القيمة تماماً أمام مكالمة هاتفية ذكية تستغل طيبة الموظف أو خوفه!».*

الهندسة الاجتماعية: هي فن التلاعب النفسي بالبشر لدفعهم إلى ارتكاب أخطاء أمنية، أو إفشاء معلومات سرية، أو منح صلاحيات دخول دون أن يشعروا بأنهم يتعرضون لهجوم.

المحركات النفسية الأربعة التي يستغلها المحتالون:

1. **الخوف والذعر (Fear):** *"حسابك البنكي سيتوقف خلال ساعتين وسيتم حجز أموالك! اضغط هنا فوراً لتفادي الإيقاف"*. الخوف يعطل التفكير المنطقي ويدفع للاندفاع.
2. **الطمع والإغراء (Greed):** *"مبروك! لقد تم اختيار رقمك للفوز بجائزة 100 ألف دولار؛ أدخل بياناتك لاستلام الشيك"*.
3. **الفضول (Curiosity):** *"شاهد الصور المسربة لزملائك في العمل في هذا الرابط السري!"*.

4. **الرغبة في المساعدة وحسن النية (Helpfulness):** ينتحل المهاجم صفة شخص محتاج أو موظف دعم فني يستغيث لمساعدته في حل مشكلة تقنية طارئة.

تذكر دائماً: **المهاجم لا يخترق أجهزتك دائماً عبر شاشات معقدة؛ بل يخترق أحياناً عواطفك وفضولك!**.

---

6. تشريح تقنيات التصيد الاحتيالي (Phishing)

التصيد هو أشهر تطبيقات الهندسة الاجتماعية اليوم، وينقسم إلى أشكال متعددة:

- **التصيد العام (Phishing):** رسائل بريد إلكتروني عشوائية تُرسل لملايين الأشخاص في نفس الوقت، تنتحل صفة شركات عالمية مثل (Netflix, Apple, PayPal).
- **التصيد الموجه (Spear Phishing):** هجوم مخصص لشخص بعينه؛ يدرس المهاجم حساباتك على وسائل التواصل، ويعرف اسم مديرك أو أفراد عائلتك، ويرسل لك رسالة تبدو صادرة من شخص تعرفه تماماً!
- **التصيد عبر الرسائل النصية القصيرة (Smishing):** رسالة SMS تدعي أنها من شركة شحن وتطلب منك تحديث عنوانك عبر رابط مشبوه لدفع رسوم طفيفة.
- **التصيد الصوتي (Vishing):** اتصال هاتفي يدعي فيه المتصل أنه من البنك المركزي أو قسم مكافحة الجرائم الإلكترونية ويطلب قراءة رمز التحقق

الذي وصلك على الهاتف.

---

7. تصنيف البرمجيات الخبيثة من منظور دفاعي

البرمجية الخبيثة (Malware): هي أي برنامج أو كود مصمم خصيصاً لإلحاق الضرر بالجهاز أو سرقة بياناته أو التجسس عليه دون موافقة صاحبه.

تنقسم البرمجيات الخبيثة إلى عائلات رئيسية:

1. **برمجيات الفدية (Ransomware):**

أخطر كوابيس العصر؛ تقوم بتشفير كافة ملفاتك وصورك وقواعد بياناتك بمفتاح تشفير معقد، وتظهر لك رسالة تطلب دفع مبالغ مالية بالعملات الرقمية مقابل إعطائك مفتاح فك التشفير.

2. **أحصنة طروادة (Trojans):**

برامج تتنكر في شكل تطبيقات مفيدة أو ألعاب مجانية أو أدوات كسر الحماية (Cracks)، وعندما تثبتها، تفتح باباً خلفياً (Backdoor) في جهازك يسمح للمهاجم بالتحكم فيه عن بُعد.

3. **برامج التجسس (Spyware):**

برمجيات تعمل في الخفاء التام لمراقبة ما تكتبه على لوحة المفاتيح (Keyloggers)، وتسجيل لقطات شاشتك، والتجسس على محادثاتك وسرقة كلمات المرور المحفوظة في المتصفح.

4. **الديدان الرقمية (Worms):**

برمجيات ذاتية التناسخ تنتشر تلقائياً عبر الشبكات المحلية من جهاز مصاب إلى الأجهزة المجاورة مستغلة ثغرات غير محدثة دون أي تدخل من المستخدم.

8. تسريب البيانات: أين تذهب أسرار الملايين؟

في كل شهر، تطالعنا الأخبار عن تسريب بيانات 50 مليون مستخدم من شركة كبرى أو متجر إلكتروني. كيف يحدث ذلك؟

غالبًا ما يحدث التسريب نتيجة:
* ثغرات في خوادم الشركة أو قواعد بياناتها تركت مفتوحة على الإنترنت بدون كلمة مرور قوية.
* هجوم تصيد نجح في الاستيلاء على حساب أحد مسؤولي النظام (System Administrators).
* هجوم على أحد الموردين الخارجيين الذين يقدمون خدمات فرعية للشركة (Supply Chain Attack).

تُباع هذه البيانات المسربة في شبكات الإنترنت المظلم (Dark Web)، وتُستخدم في شن هجمات جديدة تُعرف بـ **«حشو بيانات الاعتماد» (Credential Stuffing)**: حيث يأخذ المهاجم قائمة تضم ملايين كلمات المرور المسربة من موقع قديم، ويجربها تلقائياً ببرامج روبوتية على البنوك ومنصات البريد، آملاً أن يكون الضحية قد استخدم نفس كلمة المرور في أكثر من مكان!

9. التحديثات الأمنية: سدادات الثغرات وليست مجرد رفاهية

كثير من المستخدمين يؤجلون تحديث هواتفهم وحواسيبهم لأسابيع وشهور بحجة: "لا أريد إضاعة باقة الإنترنت" أو "الجهاز يعمل جيداً، لماذا أحدثه؟".

هذا التصرف يشبه شخصاً يرى نافذة منزله مكسورة ويقول: "لا بأس، سأصلحها العام القادم!".

عندما تكتشف الشركات ثغرة خطيرة في نظام التشغيل يستغلها المهاجمون، تسارع فوراً لبرمجة "رقعة أمنية" (Security Patch) وترسلها لك على شكل تحديث. **إذا لم تقم بالتحديث فوراً، فأنت تبقي الباب مفتوحاً لمهاجمين يعرفون بالفعل كيف يقتحمون جهازك.**

---

10. الاختبار الأمني المصرح به مقابل القرصنة غير المشروعة

يجب أن نميز تمييزاً قاطعاً بين ممارستين متباعدتين كبعد المشرق عن المغرب:

- **القرصنة غير المصرح بها (Malicious Hacking):**
أي محاولة لاختراق نظام، أو فحص شبكة، أو الدخول على حساب شخص أو مؤسسة دون إذن رسمي وصريح ومكتوب. هذا فعل مجرّم أخلاقياً وقانونياً في جميع دساتير وقوانين العالم وتترتب عليه عقوبات جنائية صارمة.

- **التقييم الأمني المصرح به (Authorized Penetration Testing):**
عمل مهني وأخلاقي نبيل، يقوم فيه مهندسو أمن مرخصون بالتعاقد مع جهة مالكة للنظام لفحص ثغراته في بيئة اختبارية مضبوطة ومحددة بدقة، بهدف كتابة تقرير سري للمؤسسة لتسد ثغراتها قبل أن يستغلها الأشرار.

المعيار الذهبي الذي يفصل بين الجريمة والعمل المهني هو كلمة واحدة: **«التصريح الخطي المسبق (Explicit Authorization)»**.

---

11. للقارئ المتخصص: نموذج التهديد STRIDE ومصفوفة MITRE ATT&CK

🔬 للقارئ المتخصص (Technical Deep Dive) في هندسة النظم وتأمين المنشآت الحساسة، يعتمد المتخصصون على أطر عمل دولية لتصنيف وتقييم التهديدات الأمنية: #### نموذج STRIDE لنمذجة التهديدات (Threat Modeling): ابتكرته شركة Microsoft ويصنف التهديدات التي تواجه أي برنامج إلى ست فئات رئيسية، كل فئة تقابل انتهاكاً لأحد معايير الأمان: | حرف النموذج | التهديد الأمني | المعيار الأمني المنتهك | الشرح الهندسي | | :---: | :--- | :--- | :---: | | **S** | انتحال الشخصية (Spoofing) | التوثيق (Authentication) | تظاهر المهاجم بأنه مستخدم أو خادم موثوق | | **T** | التلاعب بالبيانات (Tampering) | السلامة (Integrity) | تعديل غير مصرح به على البيانات في الذاكرة أو القرص أو الشبكة | |

R | إنكار الفعل (Repudiation) | عدم الإنكار (Non-Repudiation) | قدرة المهاجم على ارتكاب فعل ومحو أثره بحيث يعجز السجل عن إثباته | | **I** | تسريب المعلومات (Information Disclosure) | السرية (Confidentiality) | كشف بيانات حساسة للمستخدمين غير المصرح لهم | | **D** | حجب الخدمة (Denial of Service) | التوافر (Availability) | إغراق موارد الخادم لتعطيله عن خدمة المستخدمين الشرعيين | | **E** | تصعيد الصلاحيات (Elevation of Privilege) | التفويض (Authorization) | اكتساب مستخدم عادي لصلاحيات مسؤول النظام الكاملة (Root / Admin) |

مصفوفة MITRE ATT&CK: قاعدة معرفية عالمية توثق سلوكيات وتكتيكات المهاجمين في العالم الحقيقي (Adversary Tactics, Techniques, and Common Knowledge). تساعد فرق الدفاع السيبراني (Blue Teams) على رسم رحلة الهجوم من الاستطلاع المبدئي (Reconnaissance) واختراق الحصن الأولي (Initial Access) وصولاً إلى التسلل الجانبي داخل الشبكة (Lateral Movement) وسرقة البيانات الحيوية (Exfiltration).

---

12. ملخص الفصل الرابع

- الأمن السيبراني هو درع المجتمع لحماية الأفراد والأسر والبنى التحتية، وهو مسؤولية كل متصل بالإنترنت.

- ثالوث الأمان الكلاسيكي (CIA) يقوم على صيانة: السرية، السلامة، والتوافر.
- معادلة الخطر توضح أنك لا تستطيع إلغاء التهديدات الخارجية، ولكنك تستطيع خفض المخاطرة لسد ثغراتك الذاتية.
- الهندسة الاجتماعية تخترق السلوك البشري عبر الخوف والطمع والفضول، وتعد أسلوب الاختراق الأكثر نجاحاً اليوم.
- التحديثات الأمنية المستمرة تسد المنافذ المفتوحة في أجهزتك، وإهمالها مخاطرة غير محسوبة.
- الخط الفاصل بين العمل الأمني الأخلاقي والجريمة المعلوماتية هو الحصول على التصريح الرسمي الصريح.

---

13. اختبر معلوماتك (أسئلة للمراجعة الذاتية)

1. **اشرح ثالوث CIA مع تقديم مثال واقعي لكيفية انتهاك كل ضلع من أضلاعه الثلاثة.**
2. **ما الفارق الجوهري بين "التهديد" و"الثغرة الأمنية"؟**
3. **لماذا تركز الهندسة الاجتماعية على إثارة مشاعر الخوف والذعر لدى الضحية؟ وكيف تتصرف عندما تتلقى رسالة تهددك بإيقاف حسابك خلال دقائق؟**
4. **في نموذج STRIDE لنمذجة التهديدات، ماذا يعني مصطلح (Tampering) وما هو المعيار الأمني الذي يحمي ضده؟**

---

14. مصادر للتوسع والقراءة الموصى بها

- **NIST Cybersecurity Framework (CSF 2.0):** الإطار الرسمي للمعهد الوطني الأمريكي للمعايير والتقنية لإدارة المخاطر السيبرانية عبر `nist.gov/cyberframework`.
- **CISA Cyber Hygiene Services:** إرشادات التوعية وحماية البنى التحتية الصادرة عن وكالة الأمن السيبراني الأمريكية عبر `cisa.gov`.
- **MITRE ATT&CK Framework (`attack.mitre.org`):** الموسوعة المفتوحة الأكبر في العالم لتحليل سلوكيات المهاجمين والتكتيكات الدفاعية.

الفصل الخامس: كيف تحمي نفسك وعائلتك رقمياً؟ (الدليل العملي)

> *«الأمان الرقمي ليس برنامجاً تشتريه ببطاقتك الائتمانية وتنساه، بل هو مجموعة من العادات اليومية الواعية التي تبني بها جداراً منيعاً يحمي خصوصيتك وأحباءك».*

---

1. افتتاحية: خطة حماية المنزل التي نسيناها في العالم الرقمي

لو سألت أي شخص: *هل تخرج من بيتك وتترك الباب مفتوحاً للشارع؟ هل تضع مفتاح سيارتك معلقاً في مقبض الباب الخارجي؟ هل تترك وثائق عائلتك وأسرارك المالية ملقاة على طاولة في مقهى عام؟*
لسخر منك وقال: "بالتأكيد لا! هذا جنون مطبق!".

ومع ذلك، فإن ملايين البشر يفعلون ما يعادل ذلك تماماً في الفضاء الرقمي كل يوم:

* يستخدمون نفس كلمة المرور "السهلة" لبريدهم، وحساباتهم البنكية، وشبكات التواصل منذ سنوات!
* يضغطون على أي رابط تقع عليه أعينهم في الرسائل!
* يمنحون تطبيقات الألعاب صلاحية فتح الكاميرا وقراءة جهات الاتصال دون أن يسألوا أنفسهم: لماذا تحتاج لعبة سباق سيارات إلى قراءة أرقام هواتف عائلتي؟
* يتركون أطفالهم الصغار يبحرون بمفردهم في محيط الإنترنت دون أدنى توعية أو حماية!

هذا الفصل هو **دليلك الميداني العملي خطوة بخطوة**. لن نكتفي فيه بالنظريات، بل سنطبق أفضل المعايير الأمنية العالمية المعتمدة لتدريع حياتك وحياة عائلتك رقمياً.

---

2. هندسة كلمات المرور الحديثة وفق إرشادات NIST

لسنوات طويلة، دربتنا الشركات والمواقع على نصيحة خاطئة عفا عليها الزمن: *"اختر كلمة مرور قصيرة ومعقدة مثل* `P@$$w0rd!19` *وغيّرها كل 30 يوماً!"*.

ما النتيجة الحقيقية لهذه النصيحة القديمة؟

أصبح الناس ينسون كلماتهم المعقدة، فيكتبونها على ورقة ملصقة على الشاشة، أو يغيرون رقماً واحداً فقط (مثل جعلها `P@$$w0rd!20`)!

معايير المعهد الوطني للمعايير والتقنية الحديثة (NIST SP 800-63B):

أثبتت الأبحاث الأمنية الحديثة أن الحواسيب الخارقة تكسر الكلمات القصيرة المعقدة في ثوانٍ، بينما تعجز تماماً أمام **الكلمات الطويلة**.

لذلك، ولدت فكرة **«عبارة المرور» (Passphrase)**:
بدلاً من كلمة واحدة معقدة وصعبة الحفظ، اختر **أربع أو خمس كلمات عشوائية غير مترابطة منطقياً**، تفصل بينها مسافات أو علامات:

- **مثال ضعيف وقديم:** `J@nk!z99` (يمكن لبرامج التخمين كسرها بسرعة).
- **مثال قوي وحديث:** `نجم-شمس-فضاء-حاسوب-فارس` أو `blue-tiger-coffee-window-run`

لماذا تعد عبارة المرور أفضل بملايين المرات؟
1. سهلة جداً في التذكر بالنسبة للعقل البشري لأنها كلمات مألوفة.
2. طويلة جداً (تتجاوز 20 حرفاً)، مما يجعل كسرها عبر هجمات التخمين الشامل (Brute Force) يحتاج إلى آلاف السنين بمقاييس الحوسبة الحالية!

---

3. مدراء كلمات المرور (Password Managers): خزينتك المشفرة

السؤال المنطقي الذي يطرحه كل قارئ:
"لدي أكثر من 60 حساباً وموقعاً أستخدمها، كيف يُعقل أن أحفظ 60 عبارة مرور مختلفة وطويلة دون أن أفقد عقلي؟!".

الجواب القاطع والوحيد من خبراء الأمن السيبراني: **استخدم مدير كلمات المرور (Password Manager)**.

كيف يعمل مدير كلمات المرور؟

- تطبيق موثوق (مثل Bitwarden أو 1Password أو KeePassXC) يعمل على هاتفك وحاسوبك.
- يقوم بإنشاء كلمات مرور عشوائية فائقة القوة لكل موقع، ويحفظها في **خزينة رقمية مشفرة** بتشفير عسكري من نوع AES-256.
- **أنت لا تحفظ إلا كلمة مرور واحدة رئيسية فقط (Master Password)** لفتح هذه الخزينة.
- عندما تفتح أي موقع، يقوم التطبيق بتعبئة اسم المستخدم وكلمة المرور تلقائياً في لمح البصر.

مبدأ "انعدام المعرفة" (Zero-Knowledge Architecture):

الشركات الموثوقة لمدراء كلمات المرور تصمم خوادمها بحيث يتم تشفير وفك تشفير بياناتك **محلياً داخل هاتفك فقط**. حتى لو اقتحم مهاجم خوادم الشركة نفسها، فلن يجد سوى كتل مشفرة غير مفهومة؛ لأن مفتاح فك التشفير الوحيد موجود في عقلك أنت فقط!

---

4. المصادقة متعددة العوامل (MFA): خط الدفاع الثاني المنيع

تخيل أن كلمة مرورك قد سُربت بالفعل في اختراق لأحد المواقع؛ هل يعني هذا نهاية حسابك وسرقته فوراً؟

ليس إذا كنت قد فعّلت **«المصادقة متعددة العوامل» (Multi-Factor Authentication - MFA)**!

تقوم المصادقة على ثلاثة عناصر لإثبات هويتك:

1. **شيء تعرفه (Something you know):** مثل كلمة المرور أو رمز PIN.
2. **شيء تملكه (Something you have):** مثل هاتفك الذكي أو مفتاح أمان فيزيائي.
3. **شيء في ذاتك (Something you are):** مثل بصمة الإصبع أو بصمة الوجه.

ترتيب وسائل التحقق الثنائي من الأضعف إلى الأقوى:

1. **الرسائل النصية القصيرة (SMS Verification) - [أضعف وسيلة]:**
   رغم أنها أفضل من عدم وجود حماية مطلقاً، إلا أنها معرضة لهجمات "تبديل الشريحة" (SIM Swapping) والتنصت على شبكات الاتصالات الخلوية.
2. **تطبيقات توليد الرموز المؤقتة (Authenticator Apps) - [ممتازة وموصى بها بشدة]:**
   تطبيقات مثل (Google Authenticator, Microsoft Authenticator,

Aegis) تولد رمزاً من 6 أرقام يتغير كل 30 ثانية بدون الحاجة إلى شبكة هاتف أو إنترنت!

3. **مفاتيح الأمان الفيزيائية وتقنية مفاتيح المرور (FIDO2 / Hardware Keys & Passkeys) - [الأعلى أماناً في العالم]:**
مفاتيح صغيرة تُوصل بالهاتف أو الحاسوب (مثل YubiKey) أو تقنية Passkeys الحديثة، وهي الوسيلة الوحيدة عالمياً المقاومة تماماً للتصيد الإلكتروني (Phishing-Resistant).

---

5. حماية وتدريع البريد الإلكتروني الشخصي

بريدك الإلكتروني الأساسي ليس مجرد صندوق رسائل، بل هو **«المفتاح الرئيسي لعمارة حياتك الرقمية كلها»!**

إذا اخترق أحد بريدك الإلكتروني، فإنه يستطيع الضغط على زر "نسيت كلمة المرور" في البنك، وفي وسائل التواصل، وفي حسابك السحابي، ليستولي على كل شيء في دقائق.

القواعد الذهبية لتدريع بريدك:

- **عزل الحسابات (Account Segregation):** لا تستخدم بريدك المصرفي المهم للتسجيل في المواقع الترفيهية العشوائية ومواقع التخفيضات. اجعل لديك بريداً خاصاً بالمعاملات الحساسة، وبريداً ثانوياً للمواقع العامة.

- **تفعيل المصادقة الثنائية القوية فوراً** عبر تطبيق توثيق وتأمين خيارات استرداد الحساب.
- **مراجعة الأجهزة المتصلة باستمرار:** ادخل على إعدادات الأمان واطرد أي جلسة دخول قديمة أو جهاز غير معروف.

---

6. حماية الهواتف الذكية وضبط الصلاحيات (Permissions Hygiene)

هاتفك الذكي يعرف عنك أكثر مما يعرفه أقرب أصدقائك: يعرف موقعك الجغرافي اللحظي، ويستمع لما تقوله، ويحفظ صورك، ويسجل دقات قلبك.

كيف تحصن هاتفك؟

1. **قفل الشاشة الإلزامي:** استخدم رمز مرور لا يقل عن 6 أرقام (أو عبارة مرور أبجدية) مع البصمة البيومترية، وتجنب استخدام نمط الرسم البسيط (Pattern) الذي يسهل تخمينه من آثار الأصابع على الزجاج.
2. **حظر التثبيت من مصادر مجهولة:** لا تثبت أي تطبيق خارجي من خارج متجري Google Play و Apple App Store؛ فالغالبية الساحقة من البرمجيات الخبيثة تأتي عبر روابط خارجية ومتاجر غير رسمية.
3. **مراجعة الصلاحيات الصارمة (Permissions Review):**
   ادخل إلى إعدادات الهاتف وافحص التطبيقات واحداً تلو الآخر:
   * هل يحتاج تطبيق الآلة الحاسبة للوصول إلى جهات الاتصال؟ **(الجواب: لا!**

الغِ الصلاحية فوراً).

* اختر خيار *"أثناء استخدام التطبيق فقط"* للموقع الجغرافي بدلاً من *"السماح طوال الوقت"*.

* راجع إذن الوصول إلى (الميكروفون، الكاميرا، الصور، وسجل المكالمات).

---

7. كشف الروابط الاحتيالية والرسائل المزيفة

المحتالون محترفون في صناعة مواقع تشبه المواقع الأصلية بنسبة 99% لخداعك. فكيف تكشفهم؟

فحص عنوان النطاق (Domain Inspection):

المحتال يستطيع تزوير كل شيء في الصفحة: الشعارات، الألوان، الخطوط، ولكن **الشيء الوحيد الذي لا يستطيع تزويره هو العنوان الأصلي للموقع في شريط المتصفح!**

تأمل هذه الأمثلة على التزييف الخادع (Typosquatting):

* الموقع الأصلي للبنك: `www.alrajhibank.com.sa`
* الرابط الاحتيالي: `www.alrajhi-security-update.com`
* الرابط المزيف عبر تبديل الحروف: `www.paypal-verify.com` بدلاً من `www.paypal.com`

القاعدة الذهبية:

إذا وصلتك رسالة تطلب منك تحديث بياناتك أو دفع رسوم، **لا تنقر على الرابط**

مطلقاً؛ بل افتح متصفحك واكتب عنوان الموقع الرسمي بنفسك أو ادخل من تطبيقك المعتاد!

---

8. حماية الأطفال والمراهقين على الفضاء الرقمي

إن حماية النشء في العالم الرقمي لا تتحقق بالتجسس القمعي عليهم، بل بـ **بناء جسور الثقة والمصارحة والوعي المشترك**.

خطوات الأمان الأسري الرشيد:

1. **قاعدة المصارحة بلا عقاب:**
علم أطفالك دائماً: *"إذا رأيت شيئاً يخيفك، أو أخطأت وضغطت على رابط غريب، أو حاول شخص ابتزازك، تعال إليّ فوراً ولن أعاقبك بل سأحميك وأساعدك"*. إن خوف الطفل من عقاب والديه هو السلاح الأكبر الذي يستخدمه المبتزون لعزل الطفل وتدميره نفسياً.
2. **الرقابة الأبوية الذكية (Parental Controls):**
استخدم أدوات مثل (Google Family Link أو Apple Screen Time) لتحديد أوقات الشاشات، ومنع تثبيت التطبيقات بدون إذن الوالدين، وفلترة المحتوى الإباحي والعنيف.
3. **التوعية بمخاطر مشاركة الصور والمعلومات:**
علمهم ألا ينشروا صور غرفهم، أو اسم مدرستهم، أو جدول تدريباتهم، وألا يقبلوا إضافة أي شخص لا يعرفونه وجهاً لوجه في الحياة الحقيقية.

---

9. قاعدة النسخ الاحتياطي الذهبية (3-2-1)

تخيل أن هاتفك سقط في البحر الآن، أو أن حاسوبك أصابته برمجية فدية شفرت كل ذكريات طفولة أبنائك ومستندات عملك الهامة. ماذا تفعل؟

إذا كنت تطبق **قاعدة 3-2-1 للنسخ الاحتياطي المعتمدة عالمياً**، فستبتسم وتشتري جهازاً جديداً وتستعيد كل شيء في نصف ساعة!

```
القاعدة الذهبية (3-2-1):
[ 3 ] احتفظ بثلاث نسخ من بياناتك المهمة (النسخة الأصلية + نسختان احتياطيتان).
[ 2 ] خزن النسخ على نوعين مختلفين من وسائط التخزين (قرص صلب خارجي + خدمة سحابية).
[ 1 ] احتفظ بنسخة واحدة على الأقل في موقع جغرافي بعيد خارج منزلك (نسخة سحابية مشفرة).
```

---

10. خطط الطوارئ الفورية (Emergency Response Playbooks)

السيناريو أ: ماذا تفعل فوراً إذا فُقد أو سُرق هاتفك؟

1. **الإغلاق الفوري عن بُعد:** من أي حاسوب آخر، ادخل فوراً إلى خدمة `Find My` `iPhone` (لأجهزة أبل) أو `Find My Device` (لأجهزة أندرويد) واضغط على **"وضع الفقدان" (Lost Mode)** أو قفل ومسح الجهاز عن بُعد (Erase Device).

2. **إلغاء شريحة الاتصال (SIM Card):** اتصل بشركة الاتصالات فوراً وأوقف الشريحة حتى لا يستقبل اللص رموز التحقق للبنك عبر الـ SMS.
3. **تغيير كلمة مرور البريد الأساسي والحسابات البنكية:** واختيار تسجيل الخروج من كافة الأجهزة.

السيناريو ب: ماذا تفعل فوراً إذا شككت باختراق أحد حساباتك؟

1. **فصل الاتصال وتغيير كلمة المرور فوراً** من جهاز آخر نظيف وموثوق.
2. **تسجيل الخروج من كافة الجلسات النشطة (Log out from all sessions).**
3. **تفعيل المصادقة الثنائية (MFA)** إذا لم تكن مفعلة مسبقاً.
4. **مراجعة إعدادات الأمان:** تأكد من أن المخترق لم يضف بريداً إلكترونياً أو رقم هاتف يخصه كخيار استرداد احتياطي، وتأكد من عدم وجود قواعد إعادة توجيه (Forwarding rules) في بريدك الإلكتروني.

---

11. قائمة الفحص الذاتي للأمن الرقمي (Personal Security Checklist)

ضع علامة (✓) أمام كل معيار تطبقه حالياً لتقيس مستوى درعك الرقمي:

- [ ] 1. أستخدم عبارات مرور طويلة ومختلفة لكل حساب من حساباتي ولا أكرر كلمة مرور أبداً.

- [ ] 2. أستخدم مدير كلمات مرور موثوق يحفظ بياناتي في خزينة مشفرة.
- [ ] 3. فعلت المصادقة الثنائية (MFA) عبر تطبيق توثيق لبريدي وحساباتي البنكية والاجتماعية.
- [ ] 4. لا أشارك رموز التحقق (OTP) مع أي كائن كان، حتى لو ادعى أنه موظف البنك.
- [ ] 5. شاشة هاتفي مقفلة برمز مرور معقد لا يقل عن 6 أرقام مع القياسات البيومترية.
- [ ] 6. أحدث نظام تشغيل هاتفي وحاسوبي وتطبيقاتي بانتظام دون تأجيل.
- [ ] 7. أغلقت إمكانية تثبيت التطبيقات من مصادر مجهولة على جهازي.
- [ ] 8. راجعت صلاحيات التطبيقات وأغلقت وصول الكاميرا والميكروفون عن البرامج التي لا تحتاجهما.
- [ ] 9. أمتلك نسختين احتياطيتين من ملفاتي وصوري الهامة وفق قاعدة (3-2-1).
- [ ] 10. لا أنقر على روابط الشحنات أو تحديث البيانات في الرسائل النصية القصيرة (SMS).
- [ ] 11. أتأكد من شريط عنوان المتصفح بدقة قبل كتابة بيانات دخولي في أي موقع.
- [ ] 12. لدي بريد إلكتروني خاص ومحمي للمعاملات الرسمية منفصل عن بريد الترفيه والتسوق.
- [ ] 13. راجعت إعدادات الخصوصية في منصات التواصل وجعلت منشوراتي العائلية مقتصرة على الأصدقاء فقط.

- [ ] 14. لا أتصل بشبكات الواي فاي العامة في المقاهي لإجراء معاملات مالية أو فتح بريد العمل إلا عبر شبكة افتراضية خاصة (VPN) موثوقة.
- [ ] 15. علمت أطفالي مبدأ المصارحة التامة في حال تعرضهم لأي موقف مريب على الإنترنت.
- [ ] 16. مفعل ميزة العثور على جهازي عن بُعد وقادر على مسحه في حال السرقة.
- [ ] 17. قمت بتشفير القرص الصلب لحاسوبي الشخصي (عبر BitLocker في ويندوز أو FileVault في ماك).
- [ ] 18. أتحقق دورياً من قائمة الجلسات والأجهزة المتصلة بحساباتي وأطرد الأجهزة القديمة.
- [ ] 19. قمت بتغطية كاميرا الحاسوب بغطاء فيزيائي عند عدم الاستخدام.
- [ ] 20. أنقل هذه المعرفة لأهلي وأصدقائي تذكيراً بشعار: «المعرفة قوة، لكن قيمتها الحقيقية تتجلّى في نفع الآخرين بها».

(إذا حققت أكثر من 16 علامة: فأنت تتمتع بحصانة رقمية ممتازة تحمي بها نفسك ومجتمعك!)

---

12. للقارئ المتخصص: خوارزمية TOTP والتحقق الزمني المشفر

🔬 للقارئ المتخصص (Technical Deep Dive) كيف يستطيع تطبيق التوثيق في هاتفك توليد رمز متطابق مع رمز الخادم في نفس الثانية حتى لو كان هاتفك في وضع الطيران وبلا أي اتصال بالإنترنت؟ السر يكمن في **المعيار القياسي الدولي (RFC 6238): خوارزمية TOTP (Time-Based One-Time Password)**: #### الآلية الرياضية: 1. **المفتاح السري المشترك (K Shared Secret)**: عند تفعيل المصادقة ومسح رمز الـ QR، يتم تبادل مفتاح سري عشوائي مشفر بين هاتفك والخادم مرة واحدة فقط ويُحفظ محلياً. 2. **العداد الزمني (T)**: يتم أخذ الوقت الحالي بالثواني وفق توقيت غرينتش (Unix Epoch Time) وقسمته على نافذة زمنية مدتها 30 ثانية: $$T = \lfloor \frac{\text{Current Time} - T_0}{30} \rfloor$$. 3. **التجزئة المشفرة المبنية على الرسائل (HMAC-SHA1 / HMAC-SHA256)**: يتم حساب الـ Hash المشترك بين المفتاح السري K والعداد الزمني T: $$HS = \text{HMAC}(K, T)$$. 4. **اقتطاع الرمز الديناميكي (Dynamic Truncation)**: يتم اقتطاع 4 بايتات من ناتج الـ Hash واستخراج باقي القسمة على 10^6 لإنتاج رقم مكون من **6 خانات** يتطابق تماماً في جهازك وعلى الخادم، وينتهي مفعوله تلقائياً بعد انقضاء الـ 30 ثانية.

13. ملخص الفصل الخامس

- اختيار عبارات مرور طويلة مألوفة للإنسان وعصية على الحواسيب يمثل جوهر إرشادات الأمان الحديثة (NIST).
- مدير كلمات المرور أداة لا غنى عنها لإدارة العشرات من كلمات المرور القوية بمفتاح رئيسي واحد بهندسة انعدام المعرفة.
- المصادقة متعددة العوامل (MFA) عبر تطبيقات التوثيق أو مفاتيح FIDO2 تحمي حسابك حتى في حال تسرب كلمة المرور.
- بريدك الإلكتروني هو المفتاح الرئيسي لهويتك الرقمية ويجب عزله وتدريعه بأعلى الإجراءات الأمنية.
- حماية الأسرة والأطفال تبدأ من بناء ثقافة الحوار والمصارحة الآمنة والرقابة التقنية الرشيدة.
- تطبيق قاعدة 3-2-1 للنسخ الاحتياطي يضمن استعادة حياتك الرقمية وبياناتك الحيوية مهما بلغت جسامة الكوارث.

---

14. اختبر معلوماتك (أسئلة للمراجعة الذاتية)

1. **لماذا تفوقت "عبارة المرور" الطويلة على كلمات المرور القصيرة المعقدة في معايير NIST الحديثة؟**
2. **كيف يحميك مدير كلمات المرور الموثوق وفق مبدأ "انعدام المعرفة" (Zero-Knowledge) حتى لو تم اختراق خوادم الشركة؟**

3. **لماذا يفضل خبراء الأمن تطبيقات التوثيق (Authenticator Apps) ومفاتيح Passkeys على رسائل التحقق النصية (SMS)؟**
4. **ما هي الخطوات الثلاث الأولى التي يجب عليك اتخاذها في الدقائق الأولى من اكتشاف سرقة هاتفك الذكي؟**

---

15. مصادر للتوسع والقراءة الموصى بها

- **NIST Special Publication 800-63B:** إرشادات الهوية الرقمية وإدارة دورة حياة المصادقة وكلمات المرور عبر `pages.nist.gov/800-63-3/sp800-63b.html`.
- **FIDO Alliance Standards (`fidoalliance.org`):** المواصفات العالمية لمفاتيح الأمان الفيزيائية وتقنيات المصادقة بدون كلمات مرور (Passkeys).
- **Electronic Frontier Foundation (EFF) - Surveillance Self-Defense:** دليل مؤسسة الجبهة الإلكترونية العالمي لحماية الخصوصية الرقمية والأمان الشخصي عبر `ssd.eff.org` .

الفصل السادس: بناء البرمجيات والأنظمة الآمنة

> *«بناء برمجيات آمنة لا يعني إضافة قفل حديدي على باب خشبي متهالك، بل يعني هندسة البيت بأكمله من أساساته ليكون مقاوماً للعواصف والهزات منذ اللحظة الأولى لتصميمه».*

1. افتتاحية: البرمجة السريعة وثمن الغفلة الباهظ

في عالم تطوير البرمجيات اليوم، يدور سباق محموم حول السرعة: الشركات تريد إطلاق التطبيقات في أسواق العمل بأسرع وقت ممكن، والمطورون يتعرضون لضغوط هائلة لتسليم الميزات البرمجية (Features) قبل المواعيد المحددة.

وفي خضم هذا السباق، غالباً ما يُعامل "الأمان" كعنصر ثانوي مزعج يُؤجل إلى ما بعد الإطلاق:

"دعنا نبني التطبيق أولاً ونطلقه للناس، ثم سنفكر في تأمينه لاحقاً عندما نجد وقتاً!".

هذه العقلية هي السبب المباشر وراء كبرى الكوارث السيبرانية التي نسمع عنها يومياً في الأخبار: شركات تخسر مليارات الدولارات، وبيانات ملايين المستخدمين تُلقى على قارعة الإنترنت، وتنهار سمعة مؤسسات بُنيت على مدار عقود في لحظة واحدة!

إن قاعدة الذهب في هندسة البرمجيات تقول:
«معالجة ثغرة أمنية أثناء مرحلة التصميم والبرمجة تكلف بضعة دولارات ودقائق معدودة؛ أما معالجتها بعد وقوع الاختراق في بيئة الإنتاج فتكلف ملايين الدولارات وقد تدمر المشروع بأكمله».

في هذا الفصل، سننتقل إلى كواليس التطوير الآمن: كيف يفكر مهندس البرمجيات الواعي؟ وكيف تصمم أنظمة وبرامج تحمي بيانات مستخدميها وتلتزم بأرقى المعايير العالمية مثل **OWASP** و **NIST**؟

---

2. مبدأ الأمان منذ التصميم (Secure by Design & Default)

أطلقت وكالات الأمن السيبراني الرائدة في العالم (وعلى رأسها CISA الأمريكية) مبادرة عالمية تاريخية بعنوان: **«آمن منذ التصميم وآمن افتراضياً (Secure by Design & Secure by Default)»**.

ماذا يعني ذلك للمطور؟

1. **الأمان بالتصميم (Secure by Design):**
   أن يكون التفكير الأمني حاضراً في جلسة التخطيط الأولى: كيف ستتحرك البيانات؟ أين تُخزن؟ من يملك حق قراءتها؟ ما أسوأ سيناريو يمكن أن يحدث لو تعطل هذا الخادم؟ بدلاً من إنهاء البرمجة ثم استدعاء فريق أمني لمحاولة "ترقيع" الثغرات.
2. **الأمان افتراضياً (Secure by Default):**
   أن يأتي البرنامج فور تثبيته بأعلى إعدادات الأمان تلقائياً، دون أن تطلب من المستخدم العادي الذهاب إلى القوائم وتفعيل الحماية بنفسه؛ لأن أغلب المستخدمين لن يغيروا الإعدادات الافتراضية أبداً! (مثال: تشفير البيانات مفعّل تلقائياً، وتفعيل المصادقة الثنائية إلزامي).

---

3. القاعدة الذهبية: عدم الثقة بالمدخلات أبداً (Never Trust User Input)

إذا سألت أي خبير أمني: *ما السبب الأول لمعظم ثغرات تطبيقات الويب في العالم؟*

لسارع بالإجابة: **الثقة العمياء بمدخلات المستخدم!**

المطور الساذج يفترض أن المستخدم في خانة "الاسم" سيكتب اسمه، وفي خانة "العمر" سيكتب رقماً.

أما المهاجم، فيكتب في خانة البحث أو الاسم **أكواداً خبيثة** تستهدف إرباك قاعدة البيانات أو السيطرة على الخادم!

درعا الحماية الإلزاميان:

1. **التحقق من صحة المدخلات (Input Validation):**
   فحص المدخلات بدقة والتأكد من أنها تطابق النمط المتوقع تماماً (مثلاً: التحقق من أن حقل العمر يحتوي على أرقام صحيحة بين 1 و 120 فقط، ولا يحتوي على حروف أو رموز غريبة).
2. **التعقيم والترميز (Sanitization & Output Encoding):**
   تحويل الرموز الخاصة التي قد تفهمها لغة البرمجة أو قاعدة البيانات كأوامر (مثل الرمز `'` أو `<script>`) إلى نصوص خام غير قابلة للتنفيذ.

تذكر دائماً: **كل ما يأتي من خارج خادمك هو مدخل غير موثوق حتى تثبت سلامته بالدليل القاطع!**.

---

4. الفرق الحاسم بين المصادقة والتفويض

في كل نظام برمجي يتعامل مع مستخدمين، هناك حارسان يقفان عند الباب، ويخلط الكثيرون بين وظيفتيهما:

المفهوم	المصطلح الإنجليزي	السؤال الذي يجيب عنه	المثال البسيط
المصادقة	**Authentication (AuthN)**	**من أنت؟**	تقديم جواز سفرك أو كتابة كلمة المرور لتثبت أنك فلان.
التفويض	**Authorization (AuthZ)**	**ماذا يحق لك أن تفعل؟**	بعد إثبات هويتك، هل يحق لك دخول قمرة القيادة في الطائرة؟ أم مقعدك في الدرجة السياحية فقط؟

الخطأ البرمجي الشائع:

قد ينجح المبرمج في التحقق من أن المستخدم مسجل بالفعل (نجاح المصادقة)، لكنه ينسى فحص صلاحياته (فشل التفويض)!

فمثلاً: يقوم المستخدم العادي بتغيير الرقم في رابط المتصفح من:

`https://example.com/invoice/105` إلى

`https://example.com/invoice/106`

فإذا كان النظام رديء التصميم ولا يتحقق من أن الفاتورة رقم 106 تخص هذا المستخدم بعينه، فسيسمح له بقراءة فواتير الآخرين! تُعرف هذه الثغرة الكارثية عالمياً بـ **(BOLA / IDOR)**.

---

5. التعامل البرمجي الصحيح مع كلمات المرور: علم التجزئة (Hashing)

أكبر جريمة يرتكبها مبرمج في حق مستخدميه هي **تخزين كلمات المرور كنص صريح (Plaintext) في قاعدة البيانات!**

لو تم اختراق خادم هذا المطور، لقرأ المهاجم كلمات مرور جميع المستخدمين فوراً واستخدمها لاختراق بريدهم وبنوكهم.

ما الفرق بين التشفير والتجزئة؟

- **التشفير (Encryption) - عملية ذات اتجاهين:**
  تحويل النص الصريح إلى نص مبهم بمفتاح سري، **ويمكن فك التشفير** وإعادة النص لأصله باستخدام نفس المفتاح أو مفتاح مقابل. يُستخدم لنقل وحفظ البيانات التي نحتاج لقراءتها لاحقاً (مثل الرسائل والملفات).
- **التجزئة (Hashing) - عملية ذات اتجاه واحد فقط:**
  تمرير النص في خوارزمية رياضية تحوله إلى "بصمة رقمية" ثابتة الطول، **ويستحيل رياضياً إعادة هذه البصمة إلى كلمتها الأصلية!**

كيف يتحقق الخادم من كلمة مرورك إذاً؟

عندما تسجل دخولك، يأخذ الخادم الكلمة التي كتبتها ويجزئها بنفس المعادلة؛ فإذا تطابقت البصمتان، سمح لك بالدخول، دون أن يعرف الخادم نفسه ما هي كلمتك الأصلية!

الملح المشفر (Salt) و الفلفل (Pepper):

لمنع المهاجمين من استخدام قواميس البصمات الجاهزة (Rainbow Tables)، يضيف المطور المحترف قيمة عشوائية فريدة تُسمى **«الملح» (Salt)** إلى كل كلمة مرور قبل تجزئتها، مما يضمن أنه حتى لو استخدم شخصان نفس كلمة المرور تماماً، فستكون بصمتاهما مختلفتين كلياً في قاعدة البيانات!

ما الخوارزميات المعتمدة عالمياً لتخزين كلمات المرور؟

- **خوارزميات قديمة ومحرمة أمنياً:** MD5, SHA-1, SHA-256 (لأنها صُممت لتكون فائقة السرعة، مما يسمح للمهاجم بتجربة مليارات الاحتمالات في الثانية!).
- **الخوارزميات الدفاعية المعتمدة (بطيئة عمداً ومكلفة في الذاكرة لتكبيل المهاجمين):**
  Argon2id (الفائز بالمسابقة العالمية للتجزئة والموصى به من OWASP)، و **bcrypt**، و **PBKDF2**.

---

6. تطبيق عملي بـ Python: مقارنة برمجية بين التجزئة الخاطئة والآمنة

دعنا نرى كيف يُترجم هذا المفهوم في كود بايثون احترافي:

```
 # ========================================================
=======
# مقارنة هندسية بين التخزين الخاطئ والآمن لكلمات المرور
# كتاب الوعي الرقمي - تأليف: جنكيز
# ========================================================
======
import hashlib
import os

# --- أولاً: الممارسة الخاطئة وغير الآمنة إطلاقاً (DO NOT USE)
---
def unsafe_hash_password(password):
    """
    استخدام SHA-256 مجردة بدون ملح: ممارسة كارثية
    تسمح للمهاجم بكشف الكلمات الشائعة عبر الجداول المعكوسة
(Rainbow Tables)
    """
    return hashlib.sha256(password.encode()).hexdigest()

# --- ثانياً: الممارسة الدفاعية القياسية المعتمدة (Secure P
ractice) ---
def secure_hash_password(password):
    """
    تطبيق معيار NIST: المعتمد من PBKDF2-HMAC-SHA256
    1. توليد ملح مشفر عشوائي (Salt) بطول 16 بايت
    2. تطبيق 600,000 دورة تكرار لإبطاء هجمات التخمين
    """
    salt = os.urandom(16) # ملح عشوائي غير قابل للتوقع
    iterations = 600000   # عدد التكرارات الموصى به حديثاً
من OWASP

    derived_key = hashlib.pbkdf2_hmac(
        hash_name='sha256',
        password=password.encode('utf-8'),
        salt=salt,
        iterations=iterations
```

```
    )

    # دمج الملح مع المفتاح المشتق لتخزينهما معاً بأمان
    return salt.hex() + ":" + derived_key.hex()

def verify_password(stored_record, provided_password):
    """التحقق من صحة كلمة المرور المدخلة بمقارنة بصمتها بالمسجل"""
    salt_hex, key_hex = stored_record.split(":")
    salt = bytes.fromhex(salt_hex)

    # إعادة التجزئة بنفس الملح والتكرارات
    new_key = hashlib.pbkdf2_hmac(
        hash_name='sha256',
        password=provided_password.encode('utf-8'),
        salt=salt,
        iterations=600000
    )

    # استخدام مقارنة مقاومة للتوقيت (Timing-Attack Safe)
    import hmac
    return hmac.compare_digest(new_key.hex(), key_hex)

# تجربة حية توضح قوة الخوارزمية الدفاعية
if __name__ == "__main__":
    user_pass = "mySecretPassphrase2026!"

    # حفظ الكلمة الآمنة
    saved_record = secure_hash_password(user_pass)
    print("السجل الآمن المخزن في قاعدة البيانات:")
    print(saved_record)

    # تجربة التحقق
    is_correct = verify_password(saved_record, "mySecretPassphrase2026!")
    is_wrong = verify_password(saved_record, "WrongPassword!")
```

```
    print(f"\nنتيجة محاولة الكلمة الصحيحة: {'✓ نجح الدخول' if is_correct else '❌ فشل'}")
    print(f"نتيجة محاولة الكلمة الخاطئة : {'✓ نجح الدخول' if is_wrong else '❌ تم الصد بنجاح'}")
```

لاحظ استخدام الدالة `hmac.compare_digest`؛ فالمبرمج المتقدم يدرك أن مقارنة النصوص العادية عبر `==` تستهلك زمناً يختلف بحسب عدد الأحرف المتطابقة، مما قد يسمح للمهاجم بقياس فروق الزمن بالميكروثانية لمعرفة الحروف الصحيحة (هجمات التوقيت Timing Attacks)!

---

7. أمان قنوات الاتصال: بروتوكول TLS 1.3 وتثبيت الشهادات

عندما يتصل تطبيق الهاتف بخادم الشركة عبر شبكة إنترنت عامة، يمكن لأي شخص يتنصت على الراوتر قراءة البيانات إذا كانت مرسلة بنص صريح (عبر بروتوكول HTTP غير المشفر).

لذلك، فإن القاعدة الإلزامية في أي تطبيق حديث هي **استخدام بروتوكول HTTPS عبر طبقة النقل الآمنة (TLS - Transport Layer Security) بنسختها الحديثة TLS 1.3**.

تثبيت الشهادة الرقمية (Certificate Pinning):

في التطبيقات الحساسة (مثل تطبيقات البنوك والمحافظ الإلكترونية)، لا يكتفي المطور بـ HTTPS العادي، بل يقوم بـ "تثبيت الشهادة الرقمية" داخل كود التطبيق نفسه (SSL/Certificate Pinning)، بحيث يرفض التطبيق الحديث مع أي خادم إلا إذا قدم نفس الشهادة المشفرة المضمنة في التطبيق، مما يحبط محاولات هجمات "الرجل في المنتصف" (Man-in-the-Middle) تماماً.

---

8. مشروع أمان تطبيقات الويب المفتوح (OWASP Top 10)

يعد **مشروع أمان تطبيقات الويب المفتوح (OWASP)** المرجع العالمي الأول لمهندسي البرمجيات. تصدر المنظمة دورياً قائمة بأخطر 10 ثغرات برمجية تهدد التطبيقات في العالم:

1. **A01: انهيار التحكم في الوصول (Broken Access Control):** سماح النظام للمستخدم العادي بالوصول لبيانات مستخدمين آخرين أو لوحة الإدارة.
2. **A02: الإخفاقات التشفيرية (Cryptographic Failures):** استخدام خوارزميات قديمة أو إرسال أسرار بنص غير مشفر.
3. **A03: ثغرات الحقن (Injection - مثل SQL Injection):** إدخال أكواد خبيثة عبر حقول الإدخال لتنفيذها داخل قاعدة البيانات.

4. **A04: التصميم غير الآمن (Insecure Design):** عيوب بنيوية في منطق النظام نفسه لم تُراعَ فيها نماذج التهديد.
5. **A05: التهيئة الأمنية الخاطئة (Security Misconfiguration):** ترك كلمات المرور الافتراضية، أو تفعيل صفحات الأخطاء التفصيلية في بيئة الإنتاج.
6. **A06: المكونات القديمة والهشة (Vulnerable Components):** استخدام مكتبات خارجية مليئة بالثغرات المعلنة دون تحديث.
7. **A07: فشل المصادقة والهوية (Identification & Authentication Failures):** السماح بهجمات التخمين التلقائي دون حظر أو غياب التحقق الثنائي.
8. **A08: إخفاقات سلامة البرمجيات والبيانات (Software & Data Integrity Failures):** قبول تحديثات أو ملفات برمجية دون التحقق من توقيعها الرقمي.
9. **A09: إخفاقات التسجيل والمراقبة الأمنية (Security Logging Failures):** وقوع هجمات على النظام دون وجود سجلات ترصد الفاعل وتطلق الإنذار.
10. **A10: تزوير الطلبات من جانب الخادم (SSRF - Server-Side Request Forgery):** إجبار الخادم على جلب موارد غير مصرح بها من داخل الشبكة الداخلية.

---

9. مبدأ الامتياز الأدنى (Least Privilege) ونظام RBAC

في هندسة النظم الآمنة، يُطبق **مبدأ الامتياز الأدنى (Principle of Least Privilege)**:

«لا تمنح أي مستخدم أو برنامج أو قاعدة بيانات إلا الحد الأدنى المطلق من الصلاحيات الذي يحتاجه لإنجاز عمله فقط، ولأقصر وقت ممكن».

- تطبيق مدونة المحتوى يحتاج فقط لقراءة جدول المقالات، فلا تعطه صلاحية كتابة أو حذف على جدول المستخدمين!
- موظف خدمة العملاء يحتاج لمعرفة حالة شحن الطلب، فلا تعرض أمامه أرقام البطاقات الائتمانية للعملاء!

يُترجم هذا هندسياً عبر نظام **التحكم في الوصول القائم على الأدوار (Role-Based Access Control - RBAC)**، حيث يُعرف النظام أدواراً واضحة (زائر، كاتب، مدير مالي، مسؤول نظام)، وتُربط الصلاحيات بالأدوار بدقة متناهية.

---

10. للقارئ المتخصص: ترويسات أمان المتصفح (Security Headers)

🔬 للقارئ المتخصص (Technical Deep Dive) يمثل متصفح المستخدم خط الدفاع الأول في تطبيقات الويب. يمكن لخادم الويب فرض سياسات أمان حديدية عبر إرسال **ترويسات أمان HTTP

(Security Headers)** في كل استجابة: 1. **سياسة أمان المحتوى (Content-Security-Policy - CSP):** الترويسة الأقوى على الإطلاق للقضاء على ثغرات **Cross-Site Scripting (XSS)**. تحدد بدقة المصادر والنطاقات المسموح للمتصفح بتحميل السكربتات والصور والخطوط منها، وتحظر تماماً تنفيذ الأكواد المضمنة غير المصرح بها (Inline Scripts): `Content-Security-Policy: default-src 'self'; script-src 'self' https://trustedscripts.org;` 2. **الأمان الصارم لطبقة النقل (Strict-Transport-Security - HSTS):** تجبر المتصفح على الاتصال عبر HTTPS حصرياً وترفض محاولات الاتصال غير المشفر HTTP أو تجاوز تحذيرات الشهادات: `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` 3. **منع التضمين في إطارات خفية (X-Frame-Options):** تحمي التطبيق من هجمات اختطاف النقر **(Clickjacking)** بمنع تضمين موقعك داخل إطار `